
Cahier de Conception

Module `hrm-core`

Gestion des Ressources Humaines & Paie

Système **RT-Comops Backend**

Ce document décrit l'architecture, la modélisation et les choix de conception du module `hrm-core` au sein de la plateforme RT-Comops.

Table des matières

1	Introduction	1
1.1	Objet du document	1
1.2	Périmètre fonctionnel	1
1.3	Contexte système	1
1.4	Pile technologique	2
1.5	Conventions de nommage	2
2	Architecture générale	3
2.1	Architecture hexagonale	3
2.1.1	Couche Domaine (<code>domain/</code>)	5
2.1.2	Couche Application (<code>application/</code>)	5
2.1.3	Couche Adaptateur (<code>adapter/</code>)	5
2.1.4	Configuration (<code>config/</code>)	6
2.2	Diagramme de composants	6
2.3	Interactions inter-cores	8
2.3.1	Dépendances synchrones (ports de sortie)	10
2.3.2	Dépendances transverses (implicites)	10
2.3.3	Communication asynchrone (Kafka)	11
2.4	Propagation du contexte multi-tenant	11
2.5	Diagramme de déploiement	13
3	Cas d'utilisation	16
3.1	Vue globale des cas d'utilisation	16
3.2	Acteurs du système	19
3.3	Description détaillée des cas d'utilisation	19
3.3.1	Gestion du personnel	19
3.3.2	Gestion de la paie	22
3.3.3	Gestion des congés	24
3.3.4	Avances et prêts	25
3.3.5	Formations et évaluations	26
4	Modélisation du domaine	28
4.1	Sous-domaine Employé	28
4.1.1	Classes et responsabilités	30
4.1.2	Relations	31
4.1.3	Énumérations	32

4.2	Sous-domaine Paie	32
4.2.1	Classes et responsabilités	35
4.2.2	Relations	35
4.2.3	Énumérations	36
4.3	Sous-domaine Congés	36
4.3.1	Classes et responsabilités	38
4.3.2	Relations	38
4.3.3	Énumérations	38
4.4	Sous-domaine Formations & Évaluations	38
4.4.1	Classes et responsabilités	41
4.4.2	Relations	41
4.4.3	Énumérations	41
5	Machines à états	42
5.1	Machine à états Employee	42
5.2	Machine à états PayrollRun	44
5.3	Machine à états LeaveRequest	46
5.4	Machine à états LoanAdvance	48
6	Scénarios détaillés (Diagrammes de séquence)	51
6.1	DS-HRM-01 : Exécution du calcul de paie mensuel	51
6.1.1	Description	51
6.1.2	Acteurs et participants	51
6.1.3	Diagramme de séquence	51
6.1.4	Description textuelle du flux	53
6.2	DS-HRM-02 : Validation de la paie et publication	54
6.2.1	Description	54
6.2.2	Diagramme de séquence	54
6.2.3	Description textuelle du flux	57
6.3	DS-HRM-03 : Soumission et validation d'une demande de congé	57
6.3.1	Description	57
6.3.2	Diagramme de séquence	57
6.3.3	Phase 1 : Soumission par l'employé	59
6.3.4	Phase 2 : Approbation par le manager	59
6.4	DS-HRM-04 : Création d'un employé	59
6.4.1	Description	59
6.4.2	Diagramme de séquence	59
6.4.3	Description textuelle du flux	61
6.5	DS-HRM-05 : Demande et déduction d'une avance sur salaire	61
6.5.1	Description	61
6.5.2	Diagramme de séquence	61
6.5.3	Phase 1 : Demande par l'employé	63
6.5.4	Phase 2 : Approbation par le comptable	63

6.5.5	Phase 3 : Déduction automatique lors du calcul de paie	63
6.6	DS-HRM-06 : Acquisition mensuelle automatique des congés	63
6.6.1	Description	63
6.6.2	Diagramme de séquence	63
6.6.3	Description textuelle du flux	65
7	Diagramme d'activité	66
7.1	Processus de calcul de paie mensuel	66
7.1.1	Description du processus	69
8	Modèle logique de données	70
8.1	Vue d'ensemble	70
8.2	Description des tables	73
8.2.1	hrm_employee	73
8.2.2	hrm_contract	74
8.2.3	hrm_dependent	74
8.2.4	hrm_payroll_run	75
8.2.5	hrm_payroll_entry	76
8.2.6	hrm_payslip_line	76
8.2.7	Tables restantes	77
8.3	Contraintes d'intégrité	77
9	Règles métier et contraintes	78
9.1	Règles de calcul de paie (législation camerounaise)	78
9.2	Règles d'acquisition des congés	80
9.3	Paramètres de configuration (settings-core)	81
9.3.1	Paramètres CNPS	81
9.3.2	Paramètres IRPP	81
9.3.3	Paramètres CAC	82
9.3.4	Paramètres RAV	82
9.3.5	Paramètres CFC	82
9.3.6	Paramètres TDL	83
9.3.7	Paramètres Congés	83
9.4	Contraintes métier transverses	83
10	Glossaire	85

Table des figures

2.1	Architecture hexagonale du module hrm-core au sein de RT-Comops . .	4
2.2	Diagramme de composants interne de hrm-core	7

2.3	Cartographie des interactions entre hrm-core et les autres cores	9
2.4	Propagation du contexte Tenant / Organisation / Agence	12
2.5	Diagramme de déploiement de hrm-core dans l'infrastructure RT-Comops	14
3.1	Diagramme des cas d'utilisation globaux de hrm-core	18
4.1	Diagramme de classes Sous-domaine Employé	29
4.2	Diagramme de classes Sous-domaine Paie	34
4.3	Diagramme de classes Sous-domaine Congés	37
4.4	Diagramme de classes Sous-domaine Formations et Évaluations	40
5.1	Machine à états de l'entité Employee	42
5.2	Machine à états de l'entité PayrollRun	44
5.3	Machine à états de l'entité LeaveRequest	46
5.4	Machine à états de l'entité LoanAdvance	48
6.1	DS-HRM-01 Exécution du calcul de paie mensuel	52
6.2	DS-HRM-02 Validation de la paie et publication vers accounting-core / treasury-core	56
6.3	DS-HRM-03 Soumission et validation d'une demande de congé	58
6.4	DS-HRM-04 Création d'un employé lié à un Actor existant	60
6.5	DS-HRM-05 Demande et déduction d'une avance sur salaire	62
6.6	DS-HRM-06 Acquisition mensuelle automatique des congés	64
7.1	Diagramme d'activité Processus de calcul de paie mensuel	68
8.1	Modèle logique de données de hrm-core	72

Liste des tableaux

1.1	Technologies utilisées par hrm-core	2
2.1	Dépendances synchrones de hrm-core	10
2.2	Dépendances transverses de hrm-core	10
2.3	Événements Kafka de hrm-core	11
3.1	Acteurs du module hrm-core	19
4.1	Relations du sous-domaine Employé	31
4.2	Énumérations du sous-domaine Employé	32
4.3	Relations du sous-domaine Paie	35

4.4	Énumérations du sous-domaine Paie	36
4.5	Relations du sous-domaine Congés	38
4.6	Énumérations du sous-domaine Congés	38
4.7	Relations du sous-domaine Formations et Évaluations	41
4.8	Énumérations du sous-domaine Formations et Évaluations	41
5.1	Transitions de l'entité Employee	43
5.2	Transitions de l'entité PayrollRun	45
5.3	Transitions de l'entité LeaveRequest	47
5.4	Transitions de l'entité LoanAdvance	49
8.1	Table hrm_employee Données RH de l'employé	73
8.2	Table hrm_contract Contrats de travail	74
8.3	Table hrm_dependent Personnes à charge	74
8.4	Table hrm_payroll_run Exécution de paie	75
8.5	Table hrm_payroll_entry Ligne de paie par employé	76
8.6	Table hrm_payslip_line Lignes de bulletin de paie	76

1 Introduction

1.1 Objet du document

Le présent cahier de conception décrit l'architecture technique et fonctionnelle du module **hrm-core**, composant du système **RT-Comops Backend**. Il formalise les choix de conception, la modélisation du domaine, les interactions inter-modules et les scénarios fonctionnels détaillés.

Ce document est destiné à l'équipe de développement, aux architectes logiciels et aux parties prenantes techniques du projet.

1.2 Périmètre fonctionnel

Le module **hrm-core** couvre les domaines fonctionnels suivants :

- **Gestion du personnel** : création, modification et résiliation des employés, gestion des contrats de travail et des personnes à charge.
- **Gestion de la paie** : calcul mensuel des salaires (brut, cotisations CNPS, IRPP, CAC, net), validation, génération des ordres de paiement et déclarations CNPS.
- **Gestion des congés** : soumission des demandes, approbation par le manager, suivi des soldes, acquisition mensuelle automatique.
- **Avances et prêts sur salaire** : demande, approbation, déduction automatique sur la paie.
- **Formation et évaluation** : planification des formations, inscription des employés, évaluations de performance avec objectifs pondérés.
- **Conformité** : alertes sur les CDD expirants, suivi de la conformité réglementaire.

1.3 Contexte système

Le module **hrm-core** s'inscrit dans une architecture **modulaire monolithique** (modular monolith). L'ensemble des *cores* cohabitent dans une même JVM Spring Boot mais sont isolés par des frontières claires (interfaces, packages, événements). La

communication inter-modules se fait soit par appels synchrones (interfaces Java), soit par événements asynchrones (Apache Kafka via le pattern outbox transactionnel).

1.4 Pile technologique

TABLE 1.1 – Technologies utilisées par hrm-core

Composant	Technologie
Framework applicatif	Spring Boot (WebFlux réactif)
Accès base de données	R2DBC (réactif), Liquibase (migrations)
Base de données	PostgreSQL (un schéma par tenant)
Messagerie	Apache Kafka (topic <code>iwm.events.business</code>)
Stockage fichiers	MinIO / S3 (via <code>file-core</code>)
Sécurité	JWT (extraction par <code>kernel-core</code>), RBAC (via <code>roles-core</code>)
Programmation réactive	Project Reactor (Mono, Flux)

1.5 Conventions de nommage

Convention

- Les tables de la base de données sont préfixées par `hrm_` (ex. `hrm_employee`, `hrm_payroll_run`).
- Les événements publiés suivent la convention `ENTITE_ACTION` en majuscules (ex. `PAYROLL_VALIDATED`, `EMPLOYEE_CREATED`).
- Les endpoints REST suivent le préfixe `/api/v1/hrm/`.
- Les permissions RBAC suivent le format `hrm:<ressource>:<action>` (ex. `hrm:payroll:validate`).

2 Architecture générale

2.1 Architecture hexagonale

Le module `hrm-core` adopte une **architecture hexagonale** (Ports & Adapters), garantissant une séparation stricte entre la logique métier et les détails techniques d'infrastructure.

Architecture Hexagonale — hrm-core dans RT-Comops

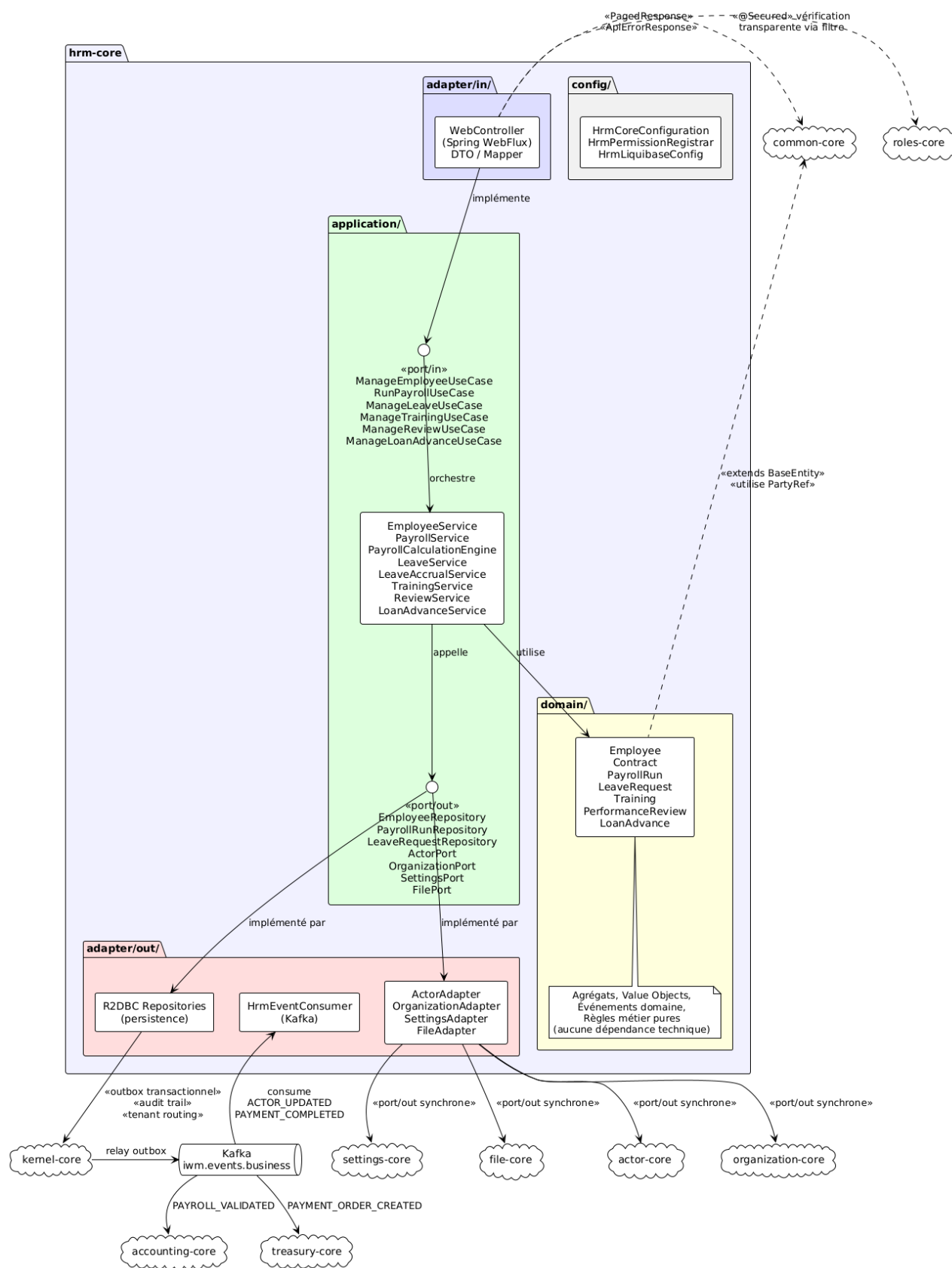


FIGURE 2.1 – Architecture hexagonale du module hrm-core au sein de RT-Comops

2.1.1 Couche Domaine (domain/)

La couche domaine contient les **agrégats**, **objets valeur** (Value Objects), **énumérations** et **règles métier pures**. Elle ne possède aucune dépendance technique (ni Spring, ni R2DBC, ni Kafka). Toutes les entités héritent de `BaseEntity` fourni par `common-core`, qui apporte :

- Les identifiants de contexte : `id` (UUID), `tenantId`, `organizationId`, `agencyId`.
- Les champs d’audit : `createdBy`, `createdAt`, `updatedBy`, `updatedAt`.
- Le verrouillage optimiste : `version` (Long).

2.1.2 Couche Application (application/)

Cette couche orchestre les cas d’utilisation. Elle est structurée en trois sous-packages :

port/in/

Interfaces des cas d’utilisation exposés (ports d’entrée). Chaque interface représente un groupe cohérent d’opérations : `ManageEmployeeUseCase`, `RunPayrollUseCase`, `ManageLeaveUseCase`, etc.

service/

Implémentations des ports d’entrée. Les services orchestrent les appels aux ports de sortie et appliquent les règles métier. Exemples : `PayrollService`, `PayrollCalculationEngine`, `LeaveAccrualService`.

port/out/

Interfaces des dépendances externes (ports de sortie). Ils abstraient l’accès aux données (`EmployeeRepository`, `PayrollRunRepository`) et aux modules voisins (`ActorPort`, `SettingsPort`, `FilePort`).

2.1.3 Couche Adaptateur (adapter/)

Les adaptateurs implémentent les ports et connectent le domaine au monde extérieur :

adapter/in/web/

Contrôleurs Spring WebFlux (endpoints REST). Ils transforment les requêtes HTTP en appels aux ports d’entrée via des DTO et des mappers. Les annotations `@Secured("hrm:...")` délèguent le contrôle d’accès à `roles-core`.

adapter/out/persistence/

Implémentations R2DBC des interfaces de repositories. Ils accèdent au schéma PostgreSQL du tenant courant.

adapter/out/integration/

Adaptateurs vers les cores voisins : `ActorAdapter`, `SettingsAdapter`, `FileAdapter`. Ces adaptateurs encapsulent les appels synchrones inter-modules.

adapter/out/messaging/

Consommateur Kafka (`HrmEventConsumer`) qui écoute les événements externes pertinents (`ACTOR_UPDATED`, `PAYMENT_COMPLETED`).

2.1.4 Configuration (config/)

- `HrmCoreConfiguration` : déclaration des beans Spring du module.
- `HrmPermissionRegistrar` : enregistrement des permissions `hrm:*` auprès de `roles-core`.
- `HrmLiquibaseConfig` : configuration des migrations Liquibase pour les tables `hrm_*`.

2.2 Diagramme de composants

Le diagramme de composants détaille la structure interne du module et le câblage entre les couches.

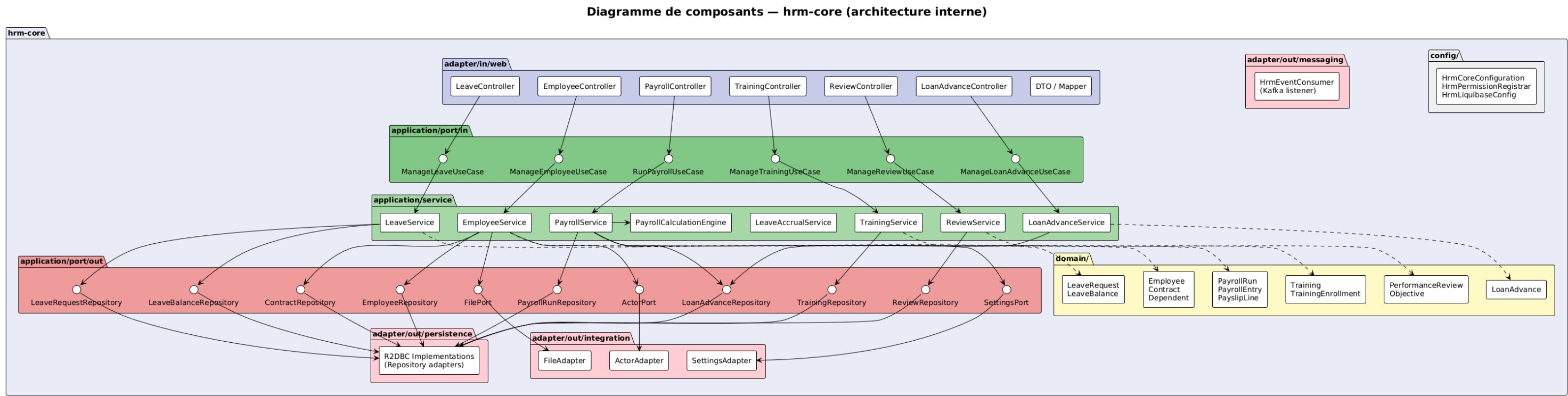


FIGURE 2.2 – Diagramme de composants interne de hrm-core

Le flux de traitement suit le chemin suivant :

1. Le **contrôleur** (adapter/in) reçoit la requête HTTP et appelle le **port d'entrée** correspondant.
2. Le **service** (application) implémente le port d'entrée, manipule les **agrégats du domaine** et invoque les **ports de sortie**.
3. Les **adaptateurs de sortie** (adapter/out) implémentent les ports de sortie : persistance R2DBC, intégration inter-cores, messaging Kafka.
4. Les **événements métier** sont publiés via le pattern outbox transactionnel de **kernel-core**.

2.3 Interactions inter-cores

Le module **hrm-core** interagit avec l'ensemble de l'écosystème RT-Comops, tant de manière synchrone qu'asynchrone.

Cartographie des interactions hrm-core ↔ autres cores

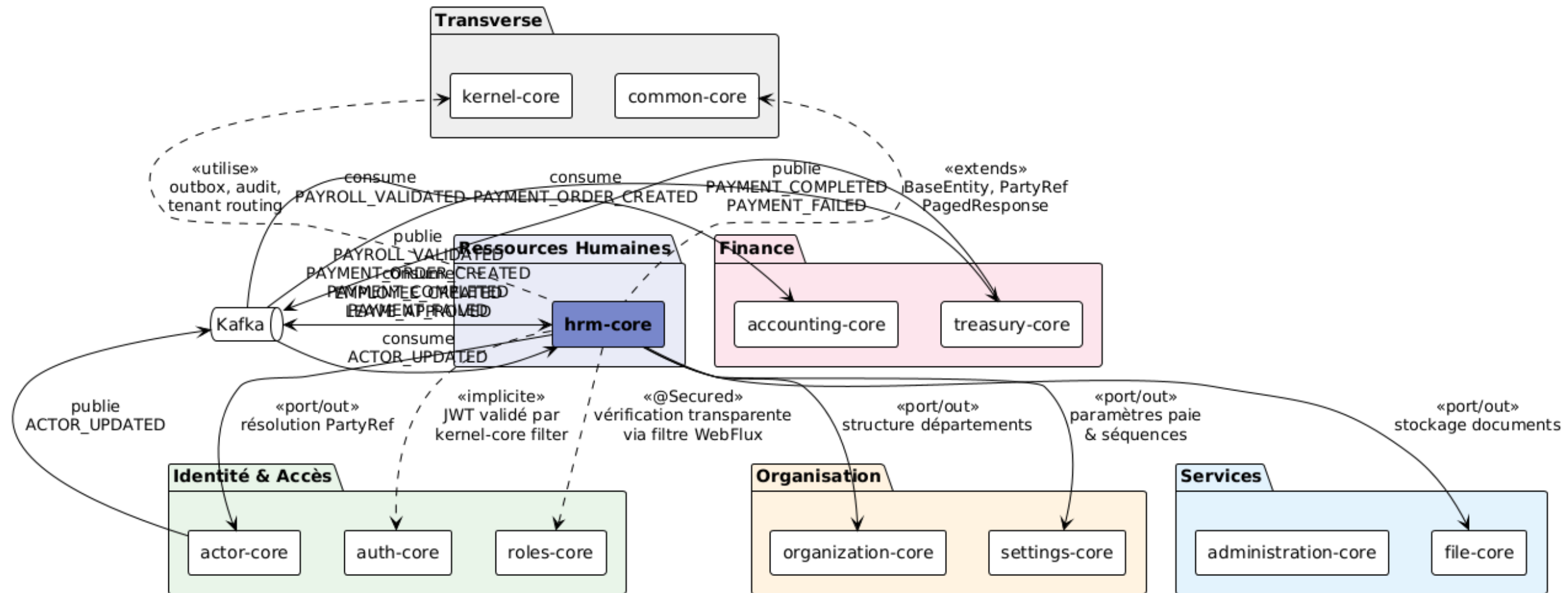


FIGURE 2.3 – Cartographie des interactions entre hrm-core et les autres cores

2.3.1 Dépendances synchrones (ports de sortie)

TABLE 2.1 – Dépendances synchrones de hrm-core

Core cible	Port	Usage
actor-core	ActorPort	Résolution des PartyRef (identité des utilisateurs/acteurs). Vérification de l'existence d'un acteur lors de la création d'un employé. Résolution du manager pour les validations.
organization-core	OrganizationPort	Consultation de la structure des départements et de la hiérarchie organisationnelle.
settings-core	SettingsPort	Lecture des paramètres de paie (taux CNPS, barème IRPP, abattement), consommation des séquences documentaires (génération du matricule). Résolution par héritage : Agence → Organisation → Tenant.
file-core	FilePort	Stockage et récupération des documents (contrats de travail, attestations de formation, justificatifs de congé) dans MinIO/S3.

2.3.2 Dépendances transverses (implicites)

TABLE 2.2 – Dépendances transverses de hrm-core

Core	Rôle
common-core	Fournit BaseEntity (audit, verrouillage optimiste), PartyRef (référence dénormalisée à un acteur), PagedResponse , ApiErrorResponse .
kernel-core	Outbox transactionnel pour la publication fiable d'événements, audit trail automatique, tenant routing (sélection du schéma PostgreSQL).
auth-core	Validation du JWT en amont (filtre WebFlux de kernel-core).
roles-core	Vérification transparente des permissions via les annotations <code>@Secured("hrm:...")</code> dans un filtre WebFlux.

2.3.3 Communication asynchrone (Kafka)

TABLE 2.3 – Événements Kafka de hrm-core

Direction	Événement	Consommateur / Producteur
Publie	PAYROLL_VALIDATED	→ accounting-core (génération des écritures comptables)
	PAYMENT_ORDER_CREATED	→ treasury-core (création des transactions bancaires/mobile money)
	EMPLOYEE_CREATED	→ notification, intégrations tierces
	LEAVE_APPROVED	→ notification, mise à jour du planning
Consomme	PAYMENT_COMPLETED	← treasury-core (confirmation de paiement)
	PAYMENT_FAILED	← treasury-core (échec de paiement)
	ACTOR_UPDATED	← actor-core (mise à jour des données identité)

Tous les événements sont publiés via le **pattern outbox transactionnel** de **kernel-core** : l'événement est inséré dans la table **kernel.outbox_event** au sein de la même transaction que la modification métier, puis relayé vers Kafka par un scheduler dédié. Cela garantit la cohérence entre l'état de la base et les événements publiés.

2.4 Propagation du contexte multi-tenant

Le système RT-Comops opère en mode **multi-tenant** avec une hiérarchie à quatre niveaux :

System → Tenant → Organisation → Agence

Propagation du contexte Tenant / Organisation / Agence

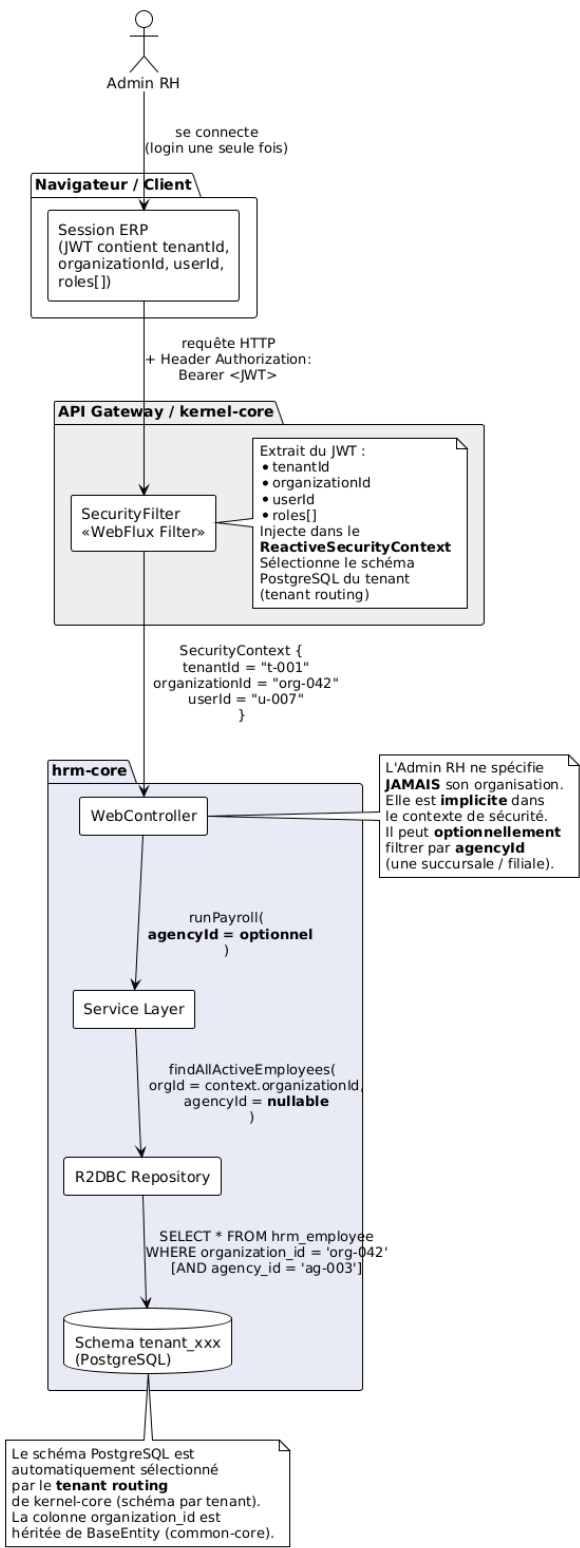


FIGURE 2.4 – Propagation du contexte Tenant / Organisation / Agence

Règle métier

- L'utilisateur ne spécifie **jamais** son `tenantId` ni son `organizationId` dans les requêtes. Ces valeurs sont extraites **implicitement** du JWT par le filtre de sécurité de `kernel-core` et injectées dans le `ReactiveSecurityContext`.
- Le `tenantId` détermine le **schéma PostgreSQL** cible (isolation des données par schéma).
- L'`organizationId` filtre implicitement toutes les requêtes de données.
- L'`agencyId` est un filtre **optionnel** que l'utilisateur peut spécifier pour restreindre le périmètre à une succursale ou filiale.

2.5 Diagramme de déploiement

Diagramme de déploiement — hrm-core dans RT-Comops

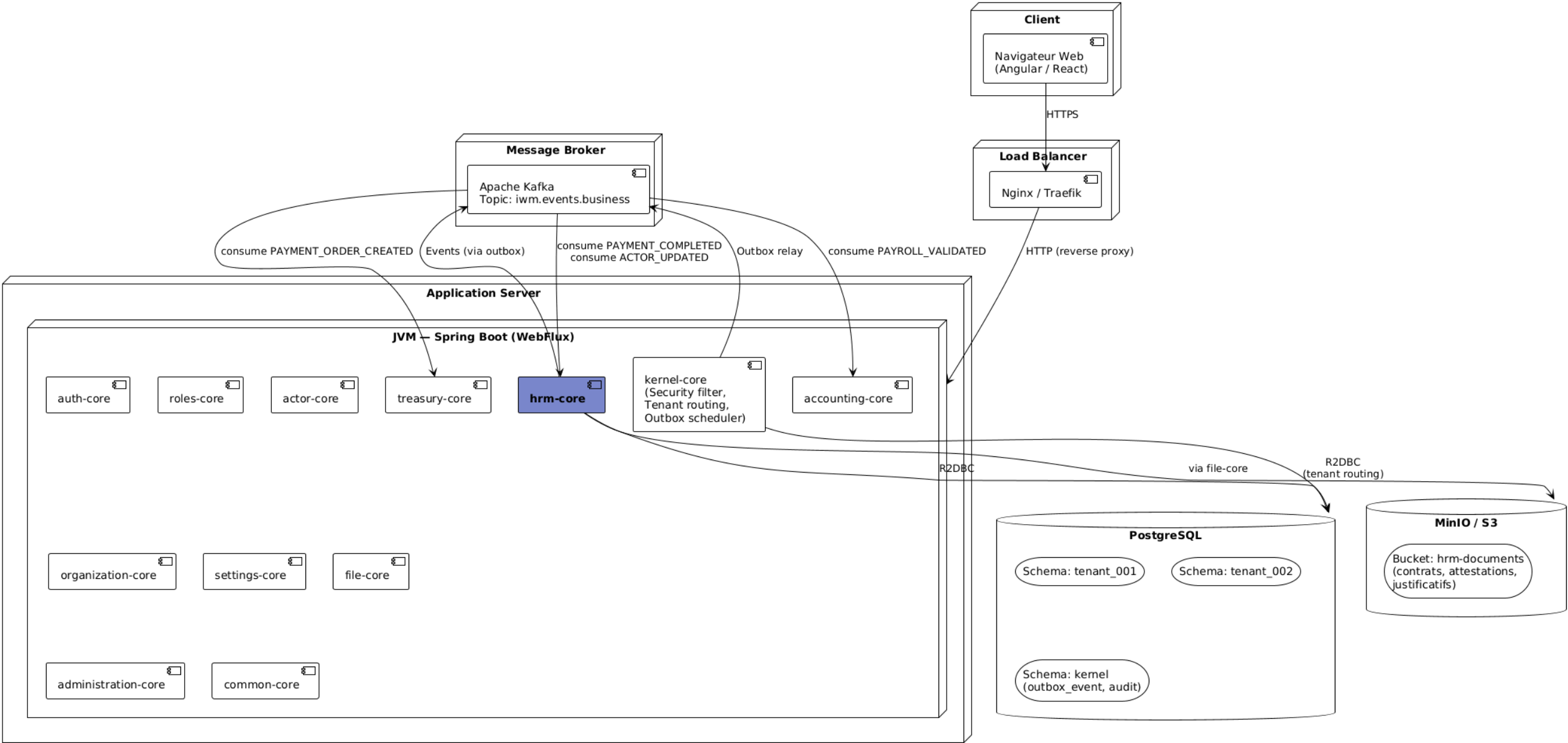


FIGURE 2.5 – Diagramme de déploiement de hrm-core dans l’infrastructure RT-Comops

L'ensemble des cores cohabitent dans une **unique JVM Spring Boot**. Le reverse proxy (Nginx/Traefik) route les requêtes HTTPS vers l'application. Les données sont stockées dans PostgreSQL (un schéma par tenant, plus un schéma `kernel` pour l'outbox et l'audit). Les fichiers sont stockés dans MinIO/S3. La communication événementielle passe par Apache Kafka.

3 Cas d'utilisation

3.1 Vue globale des cas d'utilisation

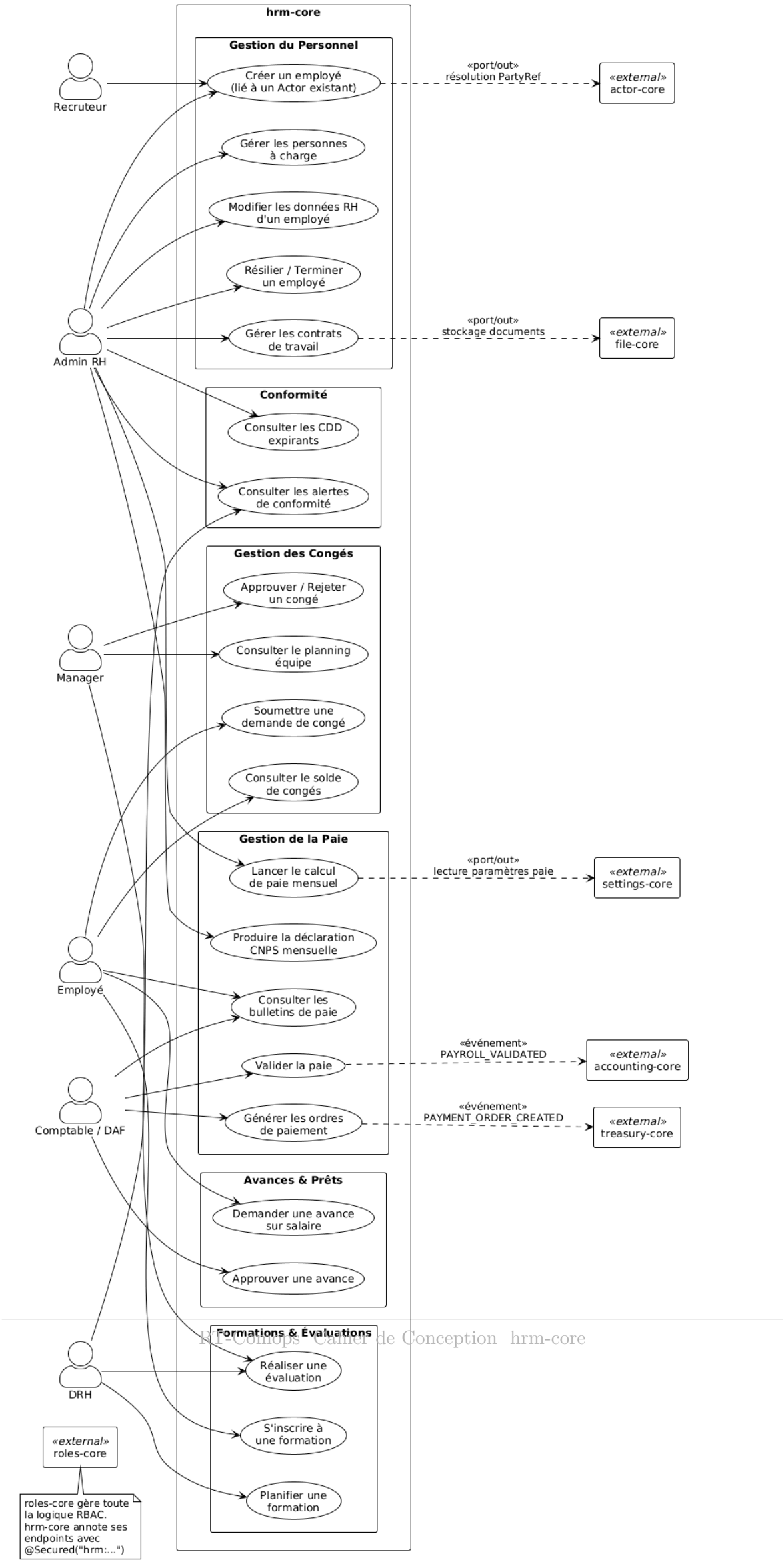


FIGURE 3.1 – Diagramme des cas d'utilisation globaux de hrm-core

3.2 Acteurs du système

TABLE 3.1 – Acteurs du module hrm-core

Acteur	Responsabilités
Admin RH	Crée et gère les employés, les contrats, les personnes à charge. Lance le calcul de paie mensuel. Gère la conformité et les alertes CDD.
Comptable / DAF	Valide la paie calculée, génère les ordres de paiement. Approuve les demandes d’avance sur salaire. Consulte les bulletins.
Manager	Approuve ou rejette les demandes de congé de son équipe. Consulte le planning de congés. Réalise les évaluations de performance.
Employé	Soumet des demandes de congé, consulte son solde. Consulte ses bulletins de paie. Demande des avances sur salaire. S’inscrit aux formations.
DRH	Planifie les formations. Participe aux évaluations. Consulte les alertes de conformité.
Recruteur	Crée un employé (lié à un Actor existant) lors de l’embauche.

3.3 Description détaillée des cas d’utilisation

3.3.1 Gestion du personnel

UC-01 : Créer un employé

Identifiant	UC-01
Nom	Créer un employé (lié à un Actor existant)
Acteurs	Admin RH, Recruteur
Préconditions	L'acteur (personne physique) existe dans <code>actor-core</code> . L'utilisateur possède la permission <code>hrm:employee:create</code> .
Scénario principal	1. L'Admin RH saisit l'<code>actorId</code> et les données RH (CNPS, catégorie, échelon, mode de paiement). 2. Le système résout la <code>PartyRef</code> via <code>actor-core</code> pour vérifier l'existence de l'acteur. 3. Le système vérifie l'unicité (pas de doublon pour cet acteur dans ce tenant). 4. Le système génère le matricule via la séquence documentaire de <code>settings-core</code>. 5. L'agrégat <code>Employee</code> est créé avec le statut <code>ACTIVE</code>. 6. Le contrat initial est créé simultanément. 7. Les soldes de congés sont initialisés (<code>ANNUAL</code>, <code>SICK</code> : <code>acquis = 0</code>, <code>pris = 0</code>). 8. Les événements <code>EMPLOYEE_CREATED</code> et <code>CONTRACT_CREATED</code> sont publiés.
Postconditions	L'employé est actif et participe aux prochains cycles de paie.
Exceptions	Acteur inexistant (404), doublon (409), données invalides (422).

UC-02 : Modifier les données RH d'un employé

Identifiant	UC-02
Nom	Modifier les données RH d'un employé
Acteurs	Admin RH
Préconditions	L'employé existe et n'est pas en statut TERMINATED .

Scénario principal	1. L'Admin RH modifie les champs éditables (catégorie, échelon, département, mode de paiement, coordonnées bancaires/mobile money). 2. Le système valide les données et persiste les modifications. 3. Un événement EMPLOYEE_UPDATED est publié.
---------------------------	---

UC-03 : Résilier un employé

Identifiant	UC-03
Nom	Résilier / Terminer un employé
Acteurs	Admin RH
Préconditions	L'employé est en statut ACTIVE ou SUSPENDED .

Scénario principal	1. L'Admin RH saisit la date de fin et le motif (démission, licenciement, fin de CDD). 2. Le contrat actif est terminé. 3. Le statut de l'employé passe à TERMINATED. 4. L'employé est exclu des cycles de paie futurs. 5. Son historique reste consultable.
---------------------------	---

UC-04 : Gérer les contrats de travail

Identifiant	UC-04
Nom	Gérer les contrats de travail
Acteurs	Admin RH
Scénario principal	1. Création d'un nouveau contrat (CDD, CDI, STAGE, INTERIM) avec salaire de base, avantages en nature, période d'essai. 2. Upload du document de contrat via file-core. 3. Renouvellement d'un CDD (extension de la date de fin). 4. Résiliation d'un contrat avec motif.
Règles	Un seul contrat actif par employé à un instant donné. Les types sont : CDD (durée déterminée), CDI (durée indéterminée), STAGE (stage), INTERIM (intérim).

UC-05 : Gérer les personnes à charge

Identifiant	UC-05
Nom	Gérer les personnes à charge
Acteurs	Admin RH
Scénario principal	1. Ajout d'une personne à charge (nom, prénom, date de naissance, lien de parenté). 2. Upload du certificat justificatif via file-core. 3. Le nombre d'enfants de moins de 14 ans impacte le calcul de l'acquisition de congés (majoration).

3.3.2 Gestion de la paie

UC-06 : Lancer le calcul de paie mensuel

Identifiant	UC-06
Nom	Lancer le calcul de paie mensuel
Acteurs	Admin RH
Préconditions	Aucun <code>PayrollRun</code> n'existe pour la même période, organisation et agence.
Scénario principal	1. L'Admin RH sélectionne la période (mois/année) et optionnellement une agence. 2. Le système charge les règles de paie depuis <code>settings-core</code>. 3. Pour chaque employé actif : récupération du contrat actif, calcul du brut, des cotisations, de l'IRPP, des réductions de prêts, du net. 4. Les bulletins (<code>PayslipLine</code>) sont générés. 5. Le <code>PayrollRun</code> est persisté avec le statut <code>CALCULATED</code>. 6. L'événement <code>PAYROLL_CALCULATED</code> est publié.
Exceptions	Doublon de période (409).

UC-07 : Valider la paie

Identifiant	UC-07
Nom	Valider la paie
Acteurs	Comptable / DAF
Préconditions	Le <code>PayrollRun</code> est au statut <code>CALCULATED</code> .
Scénario principal	1. Le comptable consulte le résumé de la paie et les bulletins individuels. 2. Il valide la paie. 3. Le statut passe à <code>VALIDATED</code>. 4. L'événement <code>PAYROLL_VALIDATED</code> est publié vers <code>accounting-core</code> (écritures comptables). 5. Des événements <code>PAYMENT_ORDER_CREATED</code> sont publiés (un par employé) vers <code>treasury-core</code>.
Alternative	Si des corrections sont nécessaires, le comptable rejette et le statut repasse à <code>DRAFT</code> .

UC-08 : Générer les ordres de paiement

Identifiant	UC-08
Nom	Générer les ordres de paiement
Acteurs	Comptable / DAF
Description	Les ordres de paiement sont générés automatiquement lors de la validation (UC-07). Chaque <code>PayrollEntry</code> produit un événement <code>PAYMENT_ORDER_CREATED</code> contenant le montant net, le canal de paiement (virement, MTN Mobile Money, Orange Money, espèces) et la référence du compte. <code>treasury-core</code> traite ces ordres et renvoie un callback <code>PAYMENT_COMPLETED</code> ou <code>PAYMENT_FAILED</code> .

3.3.3 Gestion des congés

UC-09 : Soumettre une demande de congé

Identifiant	UC-09
Nom	Soumettre une demande de congé
Acteurs	Employé
Préconditions	Le solde de congés est suffisant pour le type et la durée demandés.
Scénario principal	1. L'employé saisit le type de congé, les dates de début et de fin, et un motif. 2. Le système calcule le nombre de jours ouvrables. 3. Le système vérifie le solde disponible. 4. Le manager est résolu via <code>actor-core</code> et affecté comme valideur. 5. La demande est créée au statut <code>PENDING</code>. 6. L'événement <code>LEAVE_SUBMITTED</code> est publié.
Exceptions	Solde insuffisant (422).

UC-10 : Approuver / Rejeter un congé

Identifiant	UC-10
Nom	Approuver ou rejeter une demande de congé
Acteurs	Manager
Préconditions	La demande est au statut PENDING.
Scénario	Approbation 1. Le manager approuve la demande. 2. Le solde de congés est débité du nombre de jours. 3. Le statut passe à APPROVED. 4. Les événements LEAVE_APPROVED et LEAVE_BALANCE_UPDATED sont publiés.
Scénario	Rejet 1. Le manager rejette avec un commentaire. 2. Le statut passe à REJECTED. 3. Le solde n'est pas modifié.

3.3.4 Avances et prêts**UC-11 : Demander une avance sur salaire**

Identifiant	UC-11
Nom	Demander une avance sur salaire
Acteurs	Employé
Scénario principal	1. L'employé saisit le montant, le nombre d'échéances et le motif. 2. Le système vérifie que le cumul des prêts en cours ne dépasse pas le plafond autorisé. 3. La demande est créée au statut PENDING.
Exceptions	Plafond de prêt dépassé (422).

UC-12 : Approuver une avance

Identifiant	UC-12
Nom	Approuver une avance sur salaire
Acteurs	Comptable / DAF
Scénario principal	1. Le comptable approuve la demande. 2. Le statut passe de PENDING à IN_REPAYMENT. 3. La mensualité sera automatiquement déduite lors des prochains calculs de paie.

3.3.5 Formations et évaluations**UC-13 : Planifier une formation**

Identifiant	UC-13
Nom	Planifier une formation
Acteurs	DRH
Scénario	Le DRH crée une formation (intitulé, organisme, dates, coût, nombre de places, lieu). Le statut initial est PLANNED .

UC-14 : S'inscrire à une formation

Identifiant	UC-14
Nom	S'inscrire à une formation
Acteurs	Employé
Scénario	L'employé s'inscrit à une formation disponible. L'inscription est créée au statut ENROLLED . Après la formation, une note d'évaluation et une attestation peuvent être ajoutées.

UC-15 : Réaliser une évaluation de performance

Identifiant	UC-15
Nom	Réaliser une évaluation de performance
Acteurs	Manager, DRH
Scénario	L'évaluateur crée un bilan de performance pour un employé sur une période donnée. Il définit des objectifs pondérés, attribue des notes d'atteinte et rédige un plan d'action. L'évaluation passe par les statuts DRAFT → SUBMITTED → ACKNOWLEDGED → FINALIZED.

4 Modélisation du domaine

Ce chapitre présente les diagrammes de classes du domaine, organisés par sous-domaine fonctionnel. L'ensemble des entités héritent de **BaseEntity** (défini dans **common-core**) qui fournit les champs d'identité, de contexte multi-tenant et d'audit.

4.1 Sous-domaine Employé

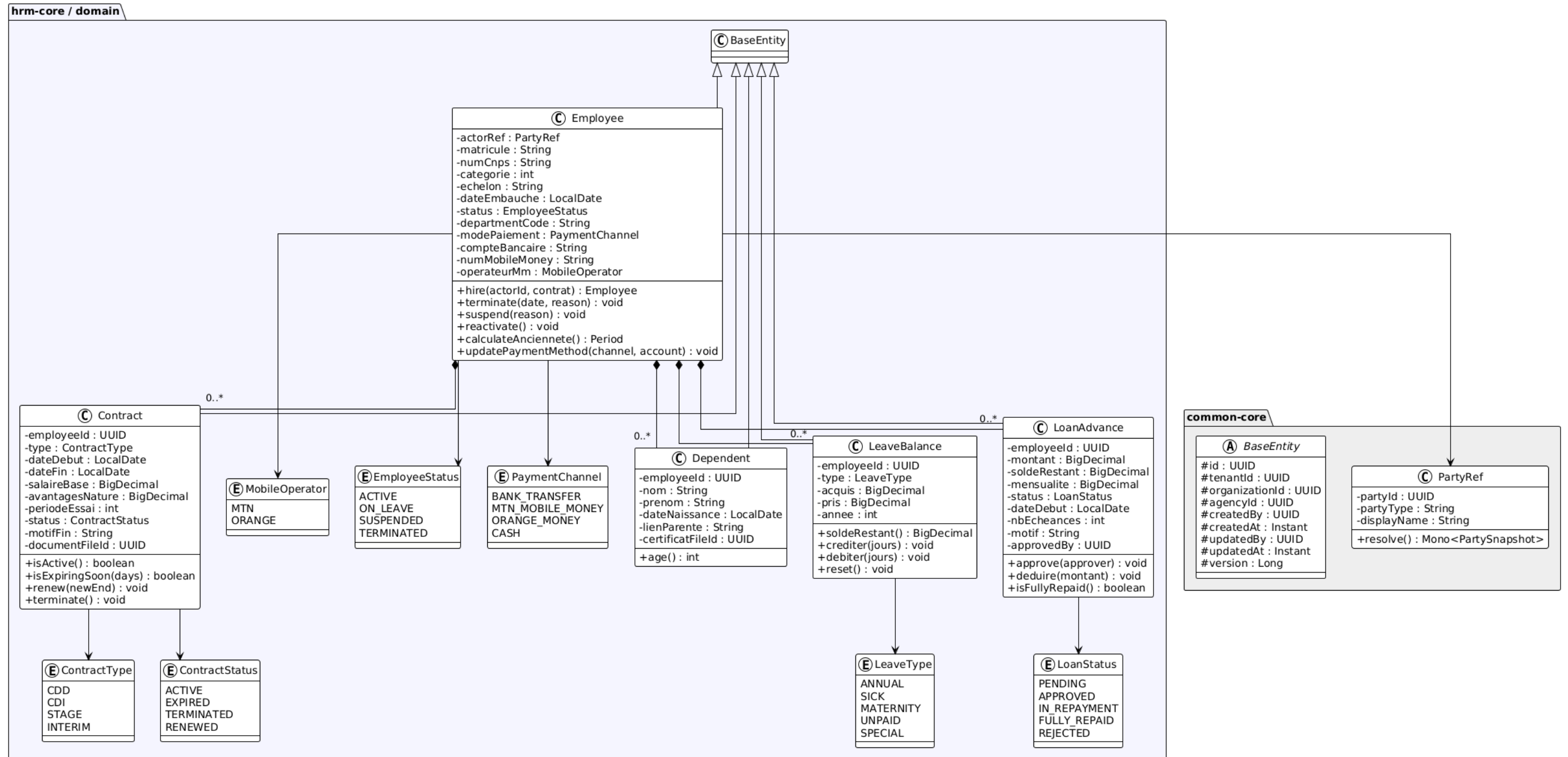


FIGURE 4.1 – Diagramme de classes Sous-domaine Employé

4.1.1 Classes et responsabilités

Employee

Agrégat racine du sous-domaine. Représente un salarié rattaché à un acteur (`PartyRef` vers `actor-core`). Porte les données RH spécifiques : matricule (généré automatiquement), numéro CNPS, catégorie professionnelle, échelon, date d'embauche, département et coordonnées de paiement. Expose les méthodes de cycle de vie : `hire()`, `terminate()`, `suspend()`, `reactivate()`.

Contract

Représente un contrat de travail. Lié à un employé par `employeeId`. Porte le type de contrat, les dates, le salaire de base, les avantages en nature et la période d'essai. La méthode `isExpiringSoon(days)` permet de détecter les CDD arrivant à échéance.

Dependent

Personne à charge d'un employé (enfant, conjoint). Le nombre d'enfants de moins de 14 ans impacte directement le calcul de l'acquisition de congés via la majoration pour charges de famille.

LeaveBalance

Solde de congés par type et par année. Suit le modèle acquis/pris avec les méthodes `crediter()`, `debiter()` et `soldeRestant()`.

LoanAdvance

Avance ou prêt sur salaire avec suivi du remboursement. La mensualité est automatiquement déduite lors du calcul de paie.

4.1.2 Relations

TABLE 4.1 – Relations du sous-domaine Employé

Relation	Cardinalité	Justification
Employee $\longrightarrow$ Contract	1..* composition	Un employé possède un historique de contrats (CDD successifs, passage en CDI). Un seul contrat est actif à un instant donné. La suppression de l'employé entraîne la suppression de ses contrats.
Employee $\longrightarrow$ Dependent	0..* composition	Un employé peut n'avoir aucune personne à charge ou en avoir plusieurs. Les personnes à charge sont liées au cycle de vie de l'employé.
Employee $\longrightarrow$ LeaveBalance	0..* composition	Un employé possède un solde de congés par type (ANNUAL, SICK, etc.) et par année. Les soldes sont créés à l'embauche et crédités mensuellement.
Employee $\longrightarrow$ LoanAdvance	0..* composition	Un employé peut avoir zéro ou plusieurs prêts/avances en cours. Le remboursement est automatique via la paie.
Employee $\longrightarrow$ PartyRef	association	Référence dénormalisée vers l'acteur (<code>actor-core</code>). Contient le <code>partyId</code> et le <code>displayName</code> pour éviter les appels systématiques à <code>actor-core</code> .

4.1.3 Énumérations

TABLE 4.2 – Énumérations du sous-domaine Employé

Enum	Valeurs et signification
EmployeeStatus	ACTIVE (en poste), ON_LEAVE (en congé), SUSPENDED (suspendu), TERMINATED (fin de contrat).
ContractType	CDD (durée déterminée), CDI (durée indéterminée), STAGE (stage), INTERIM (intérim).
ContractStatus	ACTIVE, EXPIRED, TERMINATED, RENEWED.
PaymentChannel	BANK_TRANSFER, MTN_MOBILE_MONEY, ORANGE_MONEY, CASH.
MobileOperator	MTN, ORANGE.
LeaveType	ANNUAL, SICK, MATERNITY, UNPAID, SPECIAL.
LoanStatus	PENDING, APPROVED, IN_REPAYMENT, FULLY_REPAID, REJECTED.

4.2 Sous-domaine Paie

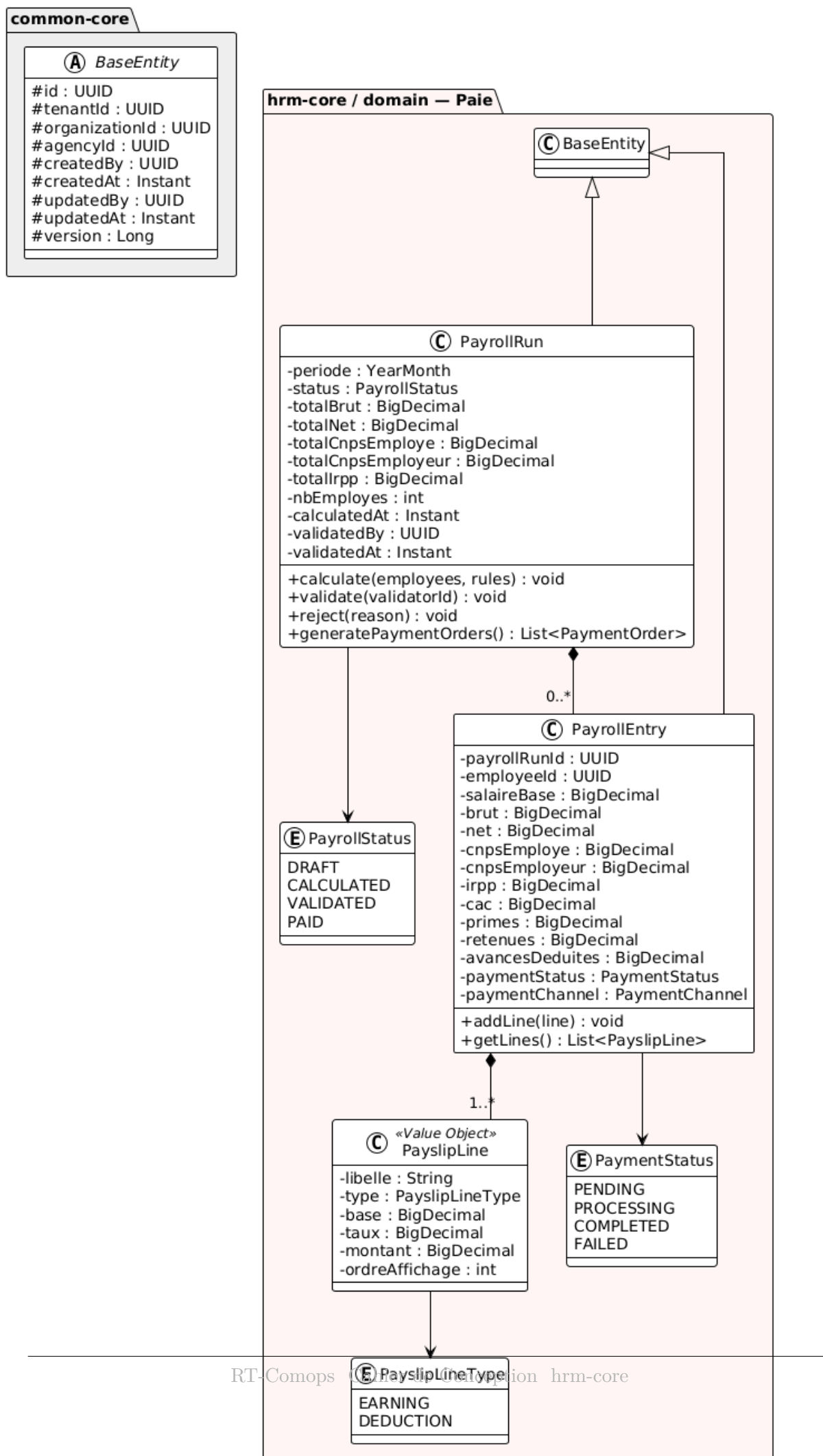


FIGURE 4.2 – Diagramme de classes Sous-domaine Paie

4.2.1 Classes et responsabilités

PayrollRun

Agrégat racine représentant une exécution de paie pour une période (mois/année), une organisation et optionnellement une agence. Porte les totaux agrégés (brut, net, cotisations) et le nombre d'employés traités. Orchestre le cycle de vie : `calculate()`, `validate()`, `reject()`, `generatePaymentOrders()`.

PayrollEntry

Ligne de paie individuelle pour un employé dans un `PayrollRun`. Détaille le salaire de base, le brut, le net, chaque cotisation (CNPS employé/employeur, IRPP, CAC), les primes, retenues et avances déduites. Porte le statut de paiement et le canal de paiement.

PayslipLine

Objet valeur représentant une ligne de bulletin de paie. Chaque ligne a un libellé, un type (gain ou retenue), une base, un taux, un montant et un ordre d'affichage.

4.2.2 Relations

TABLE 4.3 – Relations du sous-domaine Paie

Relation	Cardinalité	Justification
<code>PayrollRun</code> $\longrightarrow$ <code>PayrollEntry</code>	0..* composition	Un cycle de paie contient une entrée par employé traité. Les entrées n'existent pas indépendamment du <code>PayrollRun</code> .
<code>PayrollEntry</code> $\longrightarrow$ <code>PayslipLine</code>	1..* composition	Chaque entrée contient au minimum une ligne de bulletin (salaire de base). Les lignes détaillent les gains et retenues pour la lisibilité du bulletin.
<code>PayrollEntry</code> $\longrightarrow$ <code>Employee</code>	association	Référence vers l'employé via <code>employeeId</code> . Permet de retrouver le bulletin d'un employé donné.

4.2.3 Énumérations

TABLE 4.4 – Énumérations du sous-domaine Paie

Enum	Valeurs
PayrollStatus	DRAFT, CALCULATED, VALIDATED, PAID.
PaymentStatus	PENDING, PROCESSING, COMPLETED, FAILED.
PayslipLineType	EARNING (gain), DEDUCTION (retenue).

4.3 Sous-domaine Congés

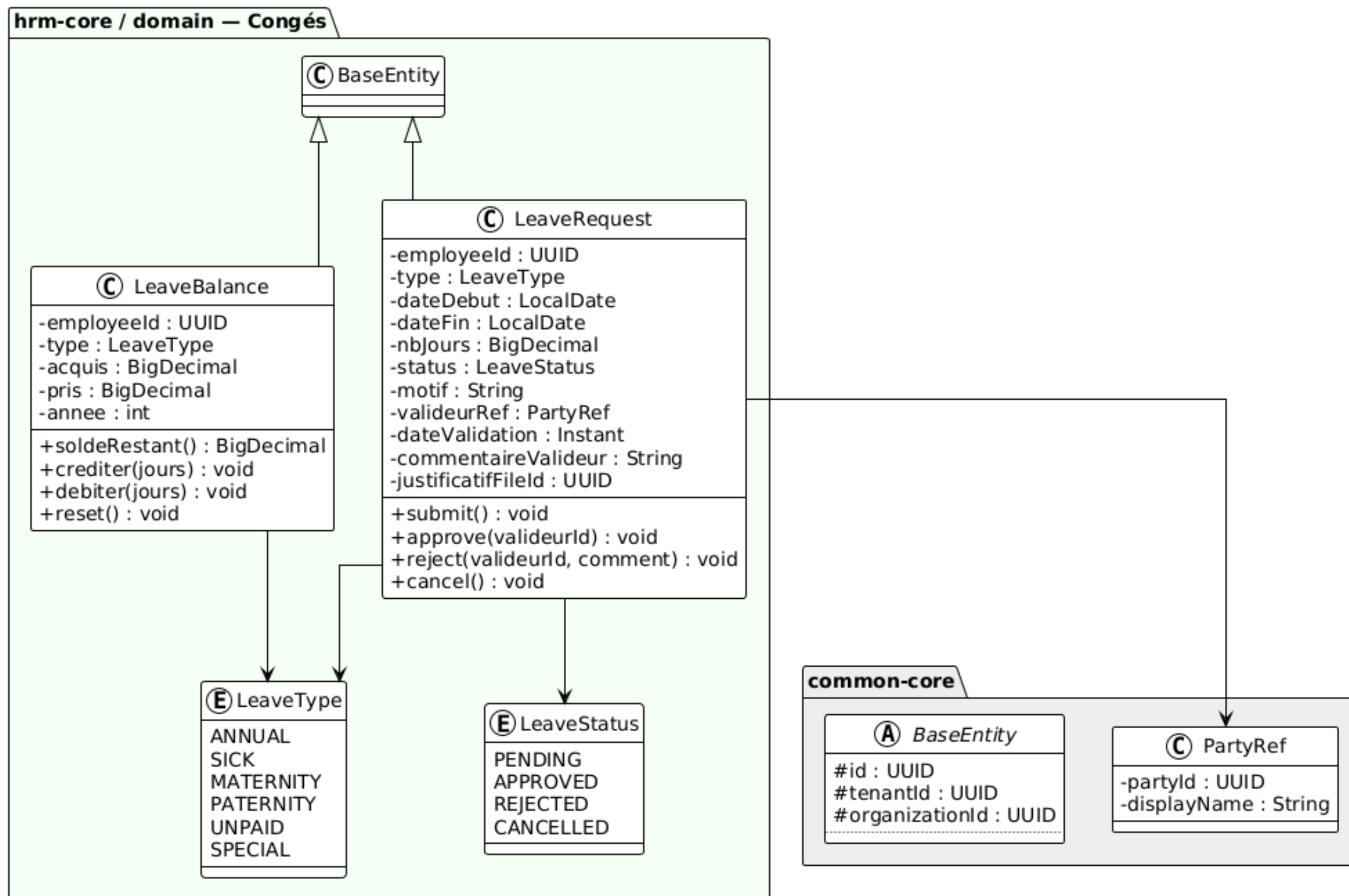


FIGURE 4.3 – Diagramme de classes Sous-domaine Congés

4.3.1 Classes et responsabilités

LeaveRequest

Demande de congé soumise par un employé. Porte le type, les dates, le nombre de jours ouvrables calculé, le motif, la référence du valideur (`PartyRef`) et le justificatif éventuel. Expose les méthodes de cycle de vie : `submit()`, `approve()`, `reject()`, `cancel()`.

LeaveBalance

Solde de congés pour un type et une année donnés. Suit l'acquisition progressive (créditée mensuellement par le `LeaveAccrualService`) et la consommation (débitée lors de l'approbation d'un congé).

4.3.2 Relations

TABLE 4.5 – Relations du sous-domaine Congés

Relation	Cardinalité	Justification
LeaveRequest $\longrightarrow$ PartyRef	association	Référence dénormalisée vers le valideur (manager). Permet d'afficher le nom du valideur sans appel systématique à <code>actor-core</code> .
LeaveRequest $\longrightarrow$ LeaveType	association	Le type de congé détermine les règles de validation (solde requis, justificatif obligatoire pour SICK, etc.).
LeaveBalance $\longrightarrow$ LeaveType	association	Un solde est tenu par type de congé. Les types ANNUAL et SICK bénéficient d'une acquisition automatique.

4.3.3 Énumérations

TABLE 4.6 – Énumérations du sous-domaine Congés

LeaveType	ANNUAL, SICK, MATERNITY, PATERNITY, UNPAID, SPECIAL.
LeaveStatus	PENDING, APPROVED, REJECTED, CANCELLED.

4.4 Sous-domaine Formations & Évaluations

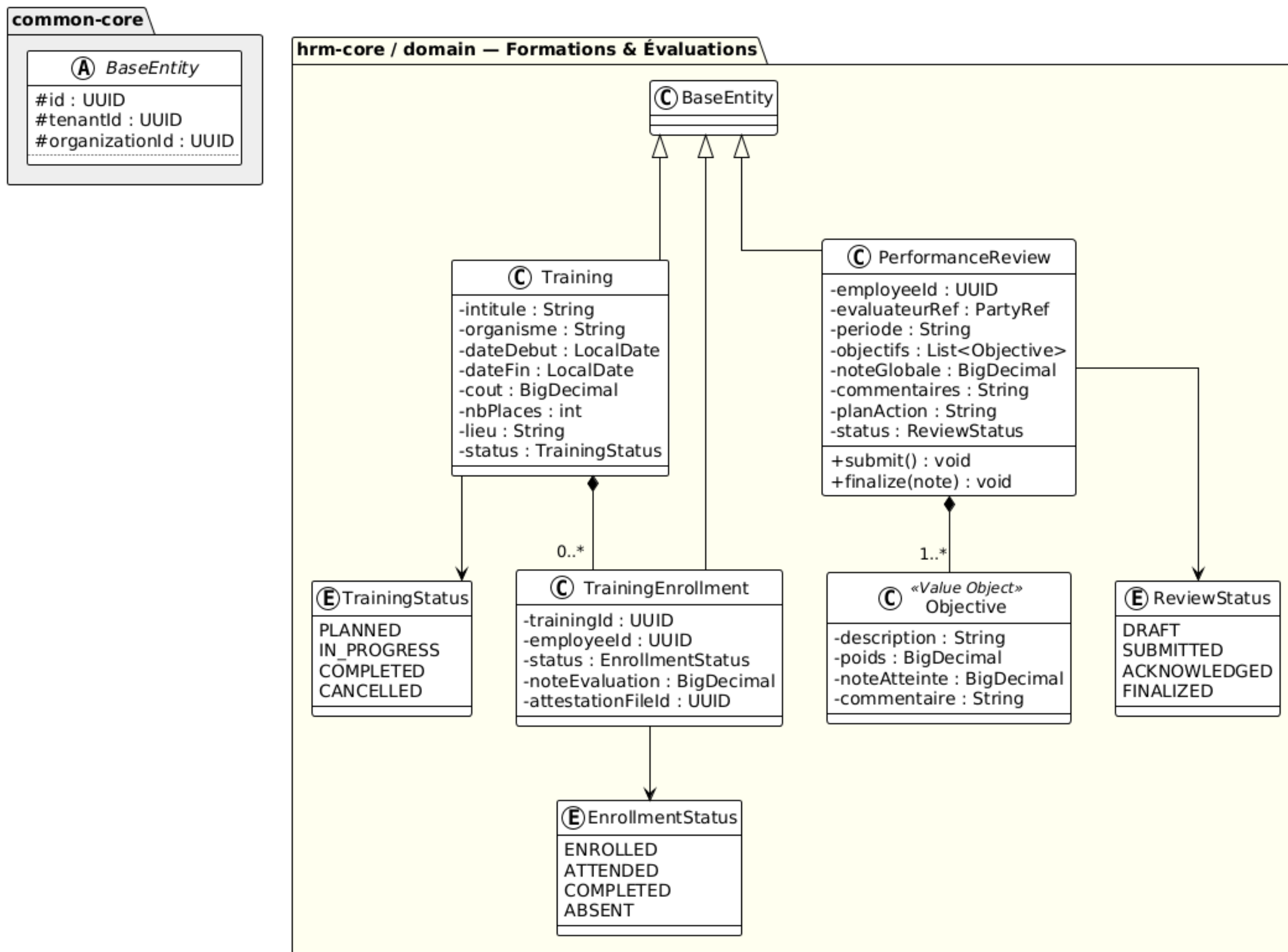


FIGURE 4.4 – Diagramme de classes Sous-domaine Formations et Évaluations

4.4.1 Classes et responsabilités

Training

Formation planifiée par le DRH. Porte l'intitulé, l'organisme formateur, les dates, le coût, le nombre de places et le lieu.

TrainingEnrollment

Inscription d'un employé à une formation. Suit le statut de participation et peut contenir une note d'évaluation et un identifiant d'attestation.

PerformanceReview

Évaluation de performance d'un employé pour une période donnée. Contient des objectifs pondérés, une note globale, des commentaires et un plan d'action.

Objective

Objet valeur représentant un objectif dans une évaluation. Chaque objectif a une description, un poids (pourcentage), une note d'atteinte et un commentaire.

4.4.2 Relations

TABLE 4.7 – Relations du sous-domaine Formations et Évaluations

Relation	Cardinalité	Justification
Training $\longrightarrow$ TrainingEnrollment	0..* composition	Une formation peut avoir plusieurs inscrits. Les inscriptions n'ont de sens que dans le contexte de leur formation.
PerformanceReview $\longrightarrow$ Objective	1..* composition	Une évaluation contient au moins un objectif. Les objectifs sont pondérés et leur somme des poids doit idéalement totaliser 100%.

4.4.3 Énumérations

TABLE 4.8 – Énumérations du sous-domaine Formations et Évaluations

TrainingStatus	PLANNED, IN_PROGRESS, COMPLETED, CANCELLED.
EnrollmentStatus	ENROLLED, ATTENDED, COMPLETED, ABSENT.
ReviewStatus	DRAFT, SUBMITTED, ACKNOWLEDGED, FINALIZED.

5 Machines à états

Ce chapitre formalise les cycles de vie des entités principales du module à travers leurs machines à états. Chaque transition est conditionnée par des règles métier précises.

5.1 Machine à états Employee

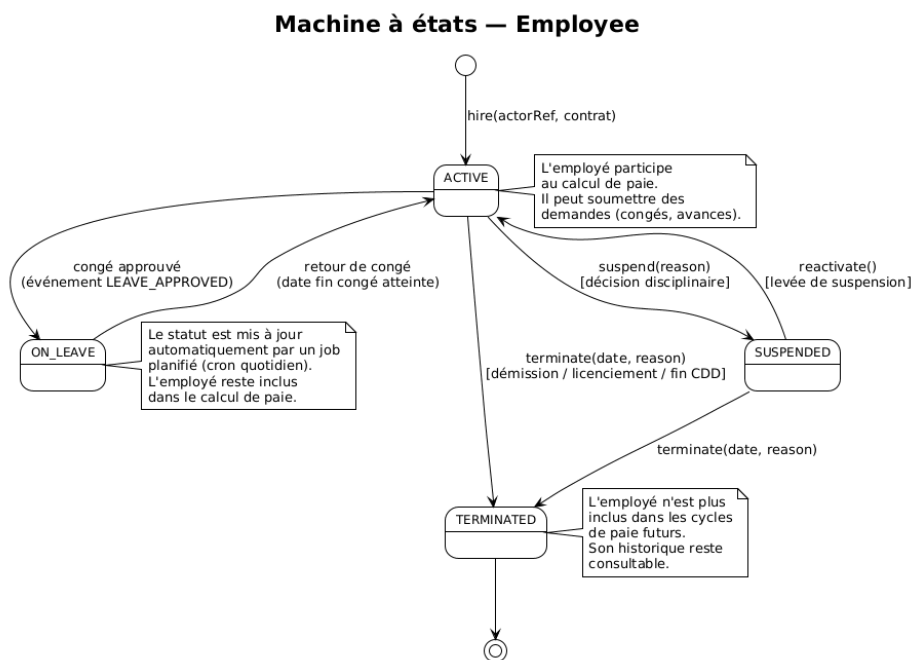


FIGURE 5.1 – Machine à états de l'entité Employee

TABLE 5.1 – Transitions de l'entité Employee

Transition	Déclencheur	Règles
[*] → ACTIVE	hire(actorRef, contrat)	Création de l'employé et de son contrat initial.
ACTIVE → ON_LEAVE	Congé approuvé	Mis à jour automatiquement par un job planifié (cron quotidien) à la date de début du congé. L'employé reste inclus dans le calcul de paie.
ON_LEAVE → ACTIVE	Retour de congé	Mis à jour automatiquement à la date de fin du congé.
ACTIVE → SUSPENDED	suspend(reason)	Décision disciplinaire. L'employé est temporairement écarté.
SUSPENDED → ACTIVE	reactivate()	Levée de suspension.
ACTIVE → TERMINATED	terminate(date, reason)	Démision, licenciement ou fin de CDD.
SUSPENDED → TERMINATED	terminate(date, reason)	Résiliation depuis l'état suspendu.

Règle métier

Un employé au statut **TERMINATED** n'est plus inclus dans les cycles de paie futurs, mais son historique (contrats, bulletins, congés) reste consultable à des fins d'archivage et d'audit.

5.2 Machine à états PayrollRun

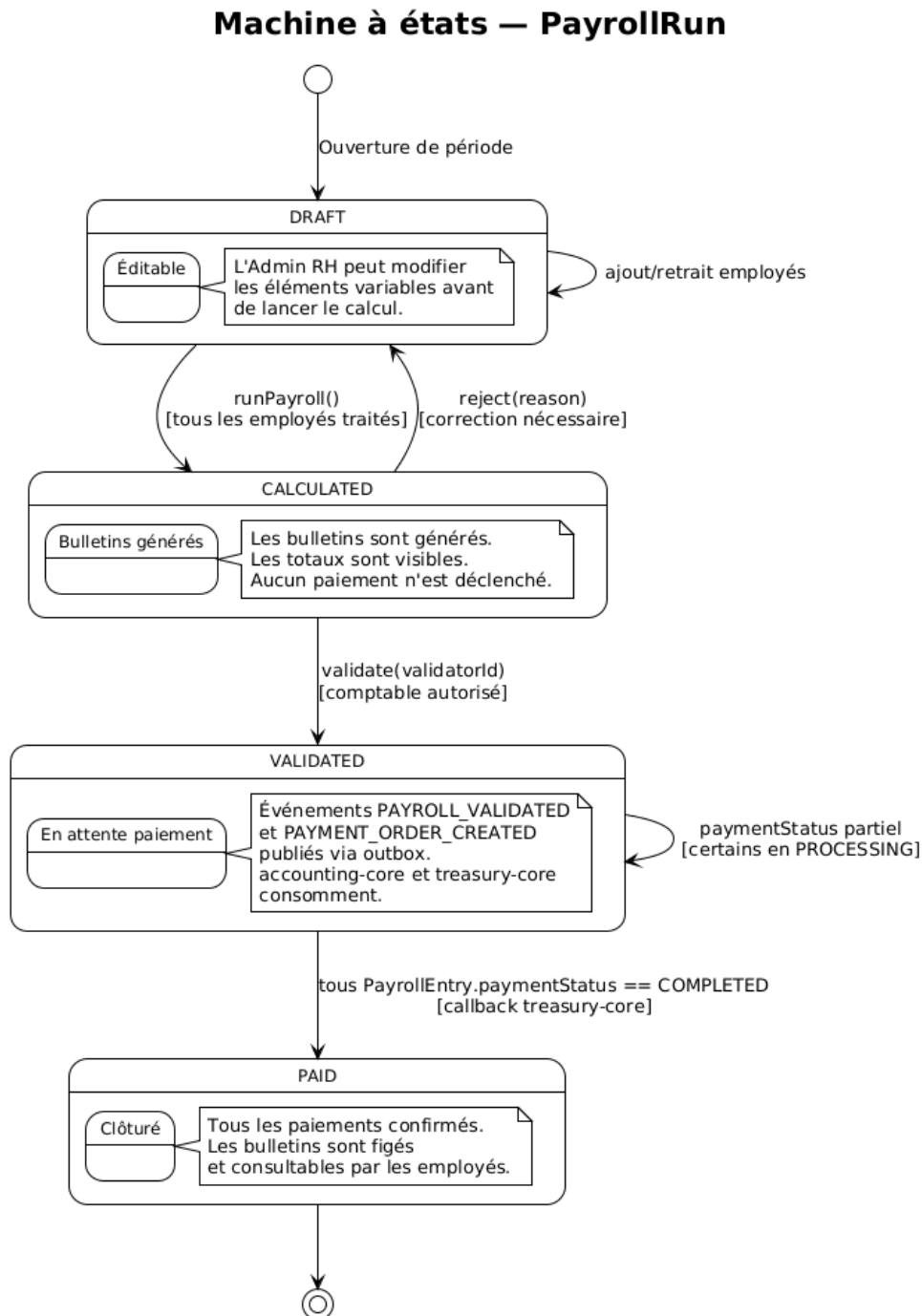


FIGURE 5.2 – Machine à états de l'entité PayrollRun

TABLE 5.2 – Transitions de l'entité PayrollRun

Transition	Déclencheur	Règles
[*] → DRAFT	Ouverture de période	L'Admin RH peut modifier les éléments variables avant de lancer le calcul.
DRAFT → CALCULATED	runPayroll()	Tous les employés actifs ont été traités. Les bulletins sont générés.
CALCULATED → VALIDATED	validate(validatorId)	Le comptable autorisé valide la paie. Les événements PAYROLL_VALIDATED et PAYMENT_ORDER_CREATED sont publiés.
CALCULATED → DRAFT	reject(reason)	Correction nécessaire, retour en mode éditable.
VALIDATED → PAID	Callback PAYMENT_COMPLETED	Tous les PayrollEntry.paymentStatus sont passés à COMPLETED (confirmé par treasury-core).

5.3 Machine à états LeaveRequest

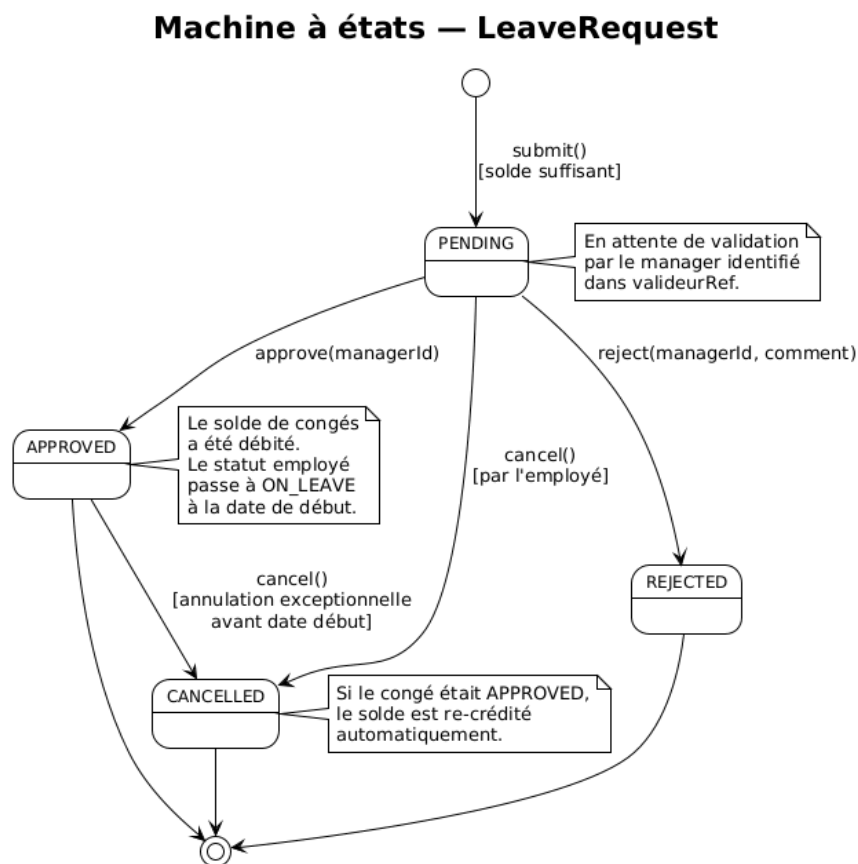


FIGURE 5.3 – Machine à états de l'entité LeaveRequest

TABLE 5.3 – Transitions de l'entité LeaveRequest

Transition	Déclencheur	Règles
[*] → PENDING	submit()	Le solde de congés doit être suffisant. Le valideur (manager) est identifié.
PENDING → APPROVED	approve(managerId)	Le solde est débité. L'employé passera à ON_LEAVE à la date de début.
PENDING → REJECTED	reject(managerId, comment)	Le solde n'est pas modifié.
PENDING → CANCELLED	cancel() par l'employé	Annulation avant validation.
APPROVED → CANCELLED	cancel() exceptionnel	Uniquement avant la date de début. Le solde est re-crédité automatiquement.

5.4 Machine à états LoanAdvance

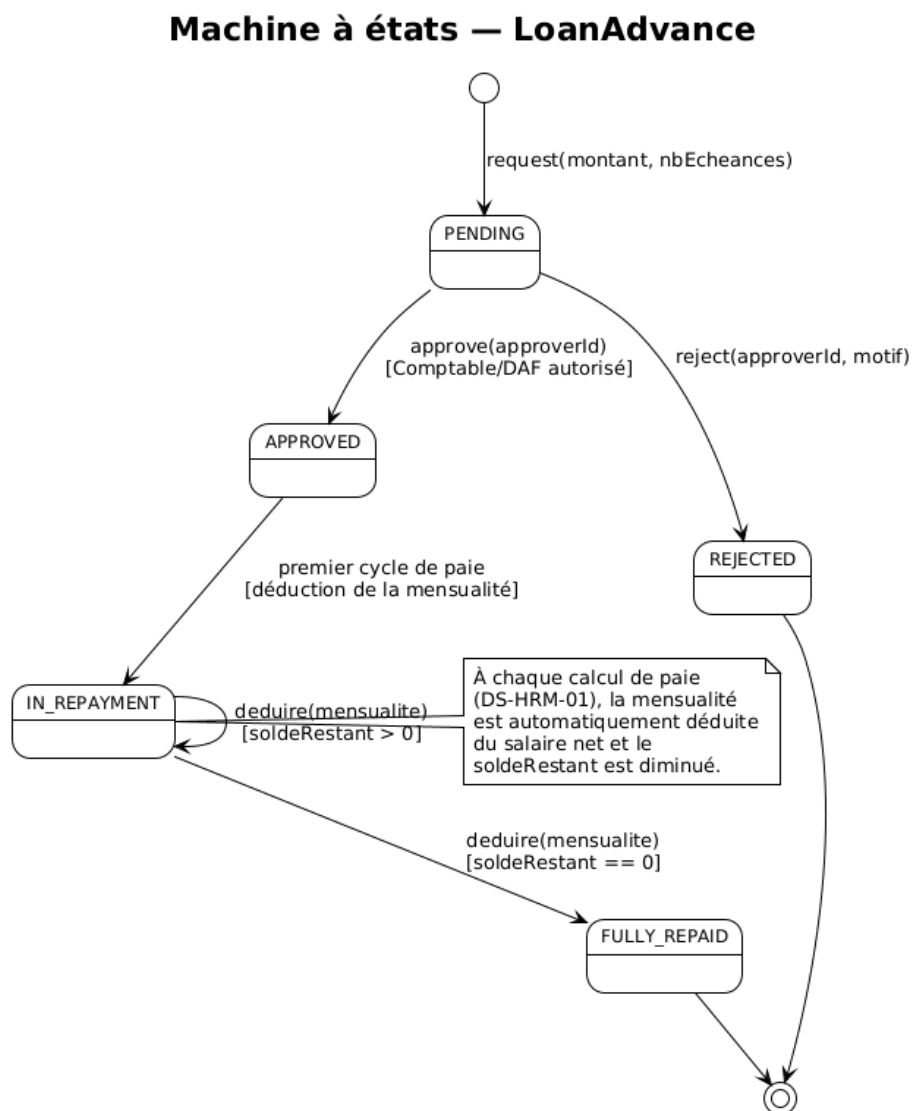


FIGURE 5.4 – Machine à états de l'entité LoanAdvance

TABLE 5.4 – Transitions de l’entité LoanAdvance

Transition	Déclencheur	Règles
[*] → PENDING	request(montant, nbEcheances)	Le cu- mul des prêts en cours ne doit pas dé- pas- ser le pla- fond.
PENDING → APPROVED	approve(approverId)	Approbation par le Comp- ta- ble/- DAF.
PENDING → REJECTED	reject(approverId, motif)	Refus avec mo- tif.
APPROVED → IN_REPAYMENT	Premier cycle de paie	La pre- mière men- sua- lité est dé- duite du sa- laire.
IN_REPAYMENT → IN_REPAYMENT	deduire(mensualite)	Tant que soldeRestant > > >

Règle métier

À chaque calcul de paie (DS-HRM-01), le `PayrollCalculationEngine` récupère les prêts actifs de chaque employé et déduit automatiquement la mensualité. La déduction est : `min(mensualite, soldeRestant)`. Lorsque le solde atteint zéro, le prêt passe en statut `FULLY_REPAID`.

6 Scénarios détaillés (Diagrammes de séquence)

Ce chapitre décrit les scénarios fonctionnels clés du module à travers des diagrammes de séquence détaillés. Chaque scénario illustre les interactions entre les acteurs, les couches de l'architecture hexagonale et les systèmes externes.

6.1 DS-HRM-01 : Exécution du calcul de paie mensuel

6.1.1 Description

Ce scénario décrit le processus complet de calcul de paie pour une période donnée. L'Admin RH déclenche le calcul ; le système charge les règles de paie, itère sur chaque employé actif, calcule les cotisations selon la législation camerounaise et génère les bulletins de paie.

6.1.2 Acteurs et participants

- **Admin RH** : déclenche le calcul via l'interface.
- **WebController** : point d'entrée REST.
- **PayrollService** : orchestre le processus.
- **PayrollCalculationEngine** : moteur de calcul pur (logique métier).
- **SettingsPort** : fournit les paramètres de paie (taux, barèmes).
- **EmployeeRepository**, **PayrollRunRepository**, **LoanAdvanceRepository** : accès aux données.
- **OutboxEvent** : publication des événements via le pattern outbox.

6.1.3 Diagramme de séquence

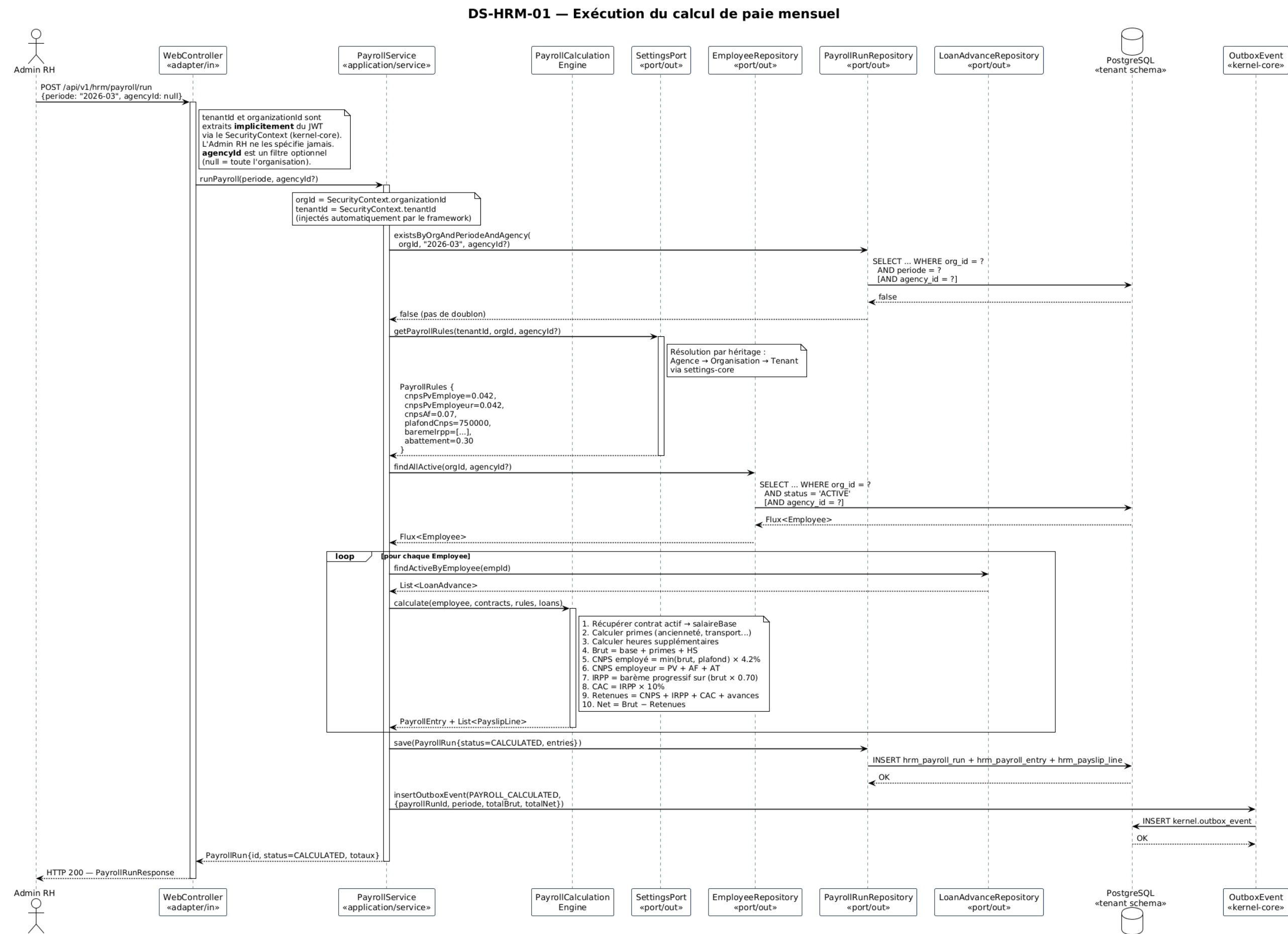


FIGURE 6.1 – DS-HRM-01 Exécution du calcul de paie mensuel

6.1.4 Description textuelle du flux

1. L'Admin RH envoie une requête POST `/api/v1/hrm/payroll/run` avec la période (ex. 2026-03) et optionnellement un `agencyId`. Le `tenantId` et l'`organizationId` sont extraits implicitement du JWT.
 2. Le `PayrollService` vérifie qu'aucun `PayrollRun` n'existe déjà pour cette combinaison période/organisation/agence. En cas de doublon, une erreur HTTP 409 est retournée.
 3. Les règles de paie sont chargées depuis `settings-core` avec résolution par héritage (Agence → Organisation → Tenant). Les paramètres incluent :
 - Taux CNPS PV employé : 4,2%
 - Taux CNPS PV employeur : 4,2%
 - Taux CNPS AF : 7%
 - Plafond CNPS : 750 000 FCFA
 - Barème IRPP progressif
 - Abattement forfaitaire : 30%
 4. Le service récupère tous les employés au statut `ACTIVE` pour l'organisation (et l'agence si spécifiée).
 5. **Pour chaque employé**, le `PayrollCalculationEngine` effectue le calcul suivant :
 - a. Récupération du contrat actif → `salaireBase`.
 - b. Calcul des primes (ancienneté, transport, etc.).
 - c. Calcul des heures supplémentaires.
 - d. **Brut** = base + primes + HS + avantages en nature.
 - e. **CNPS employé** = $\min(\text{brut}, \text{plafond}) \times 4,2\%$.
 - f. **CNPS employeur** = PV + AF + AT.
 - g. **Net imposable** = $\text{brut} \times 70\%$ (après abattement de 30%).
 - h. **IRPP** = application du barème progressif camerounais sur le net imposable.
 - i. **CAC** = $\text{IRPP} \times 10\%$.
 - j. Récupération des prêts actifs → mensualités à déduire.
 - k. **Retenues totales** = CNPS + IRPP + CAC + prêts.
 - l. **Net à payer** = Brut – Retenues totales.
- Un `PayrollEntry` et ses `PayslipLine` sont générés.
6. Le `PayrollRun` est persisté avec le statut `CALCULATED` et les totaux agrégés.
 7. L'événement `PAYROLL_CALCULATED` est inséré dans la table outbox.
 8. La réponse HTTP 200 est retournée avec le résumé de la paie.

6.2 DS-HRM-02 : Validation de la paie et publication

6.2.1 Description

Ce scénario décrit la validation de la paie par le comptable et la publication des événements vers **accounting-core** (écritures comptables) et **treasury-core** (ordres de paiement), ainsi que le callback asynchrone de confirmation de paiement.

6.2.2 Diagramme de séquence

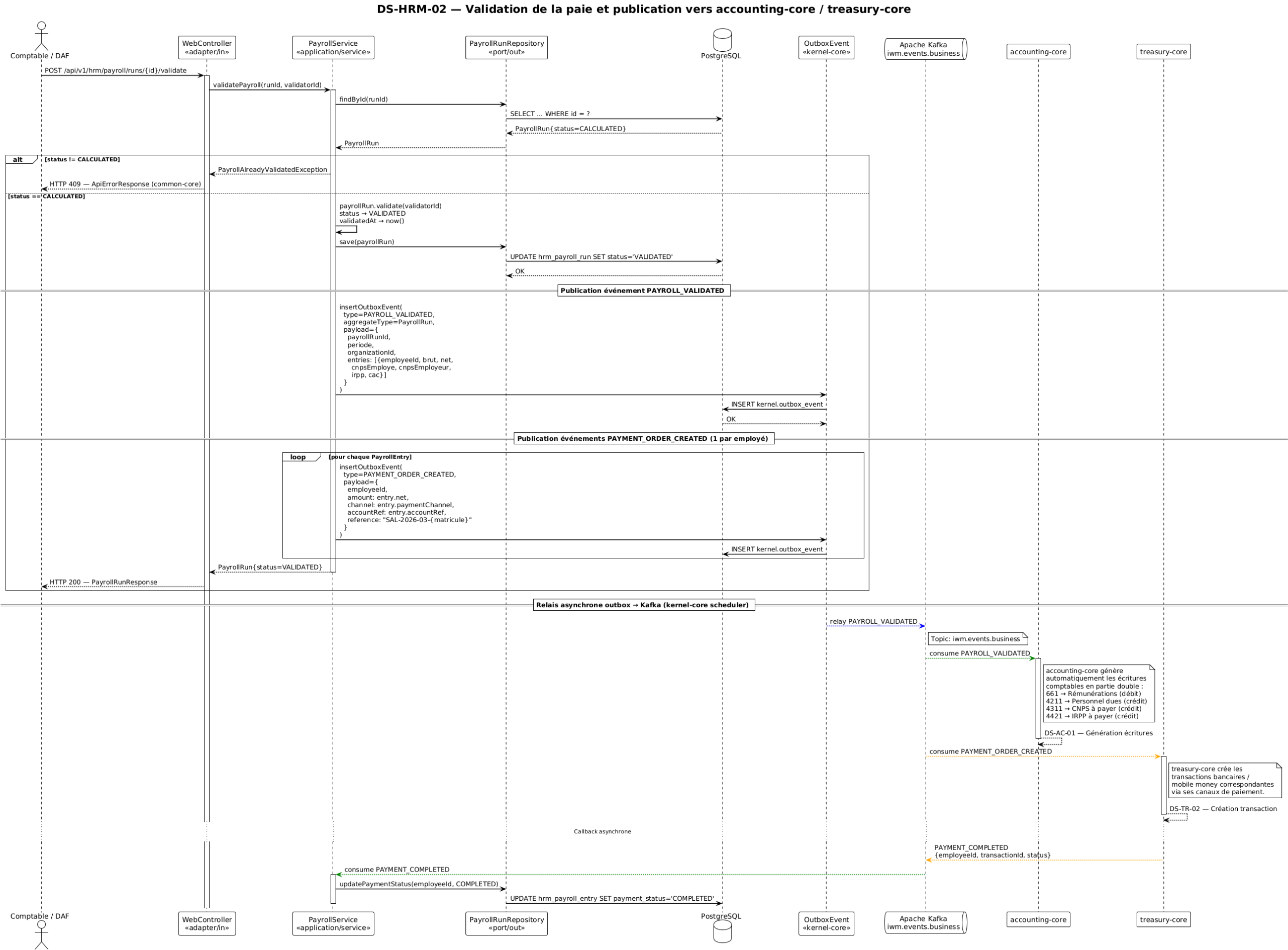


FIGURE 6.2 – DS-HRM-02 Validation de la paie et publication vers accounting-core / treasury-core

6.2.3 Description textuelle du flux

1. Le Comptable/DAF envoie `POST /api/v1/hrm/payroll/runs/{id}/validate`.
2. Le `PayrollService` vérifie que le `PayrollRun` est au statut `CALCULATED`. Si ce n'est pas le cas, une erreur HTTP 409 est retournée.
3. Le statut passe à `VALIDATED`. Les champs `validatedBy` et `validatedAt` sont renseignés.
4. Un événement `PAYROLL_VALIDATED` est publié via l'outbox. Le payload contient l'identifiant du run, la période, l'organisation et le détail par employé (brut, net, cotisations). `accounting-core` consomme cet événement et génère automatiquement les écritures comptables en partie double :
 - 661 Rémunérations du personnel (débit)
 - 4211 Personnel, rémunérations dues (crédit)
 - 4311 CNPS à payer (crédit)
 - 4421 IRPP à payer (crédit)
5. Pour chaque `PayrollEntry`, un événement `PAYMENT_ORDER_CREATED` est publié. Le payload contient le montant net, le canal de paiement, la référence du compte et une référence de paiement (ex. `SAL-2026-03-{matricule}`). `treasury-core` crée les transactions bancaires ou mobile money correspondantes.
6. **Callback asynchrone** : lorsque `treasury-core` confirme le paiement, il publie un événement `PAYMENT_COMPLETED` (ou `PAYMENT_FAILED`). Le `HrmEventConsumer` consomme cet événement et met à jour le `paymentStatus` de la `PayrollEntry` correspondante. Lorsque toutes les entrées sont `COMPLETED`, le `PayrollRun` passe au statut `PAID`.

6.3 DS-HRM-03 : Soumission et validation d'une demande de congé

6.3.1 Description

Ce scénario en deux phases illustre le cycle complet d'une demande de congé : soumission par l'employé, puis approbation (ou rejet) par le manager.

6.3.2 Diagramme de séquence

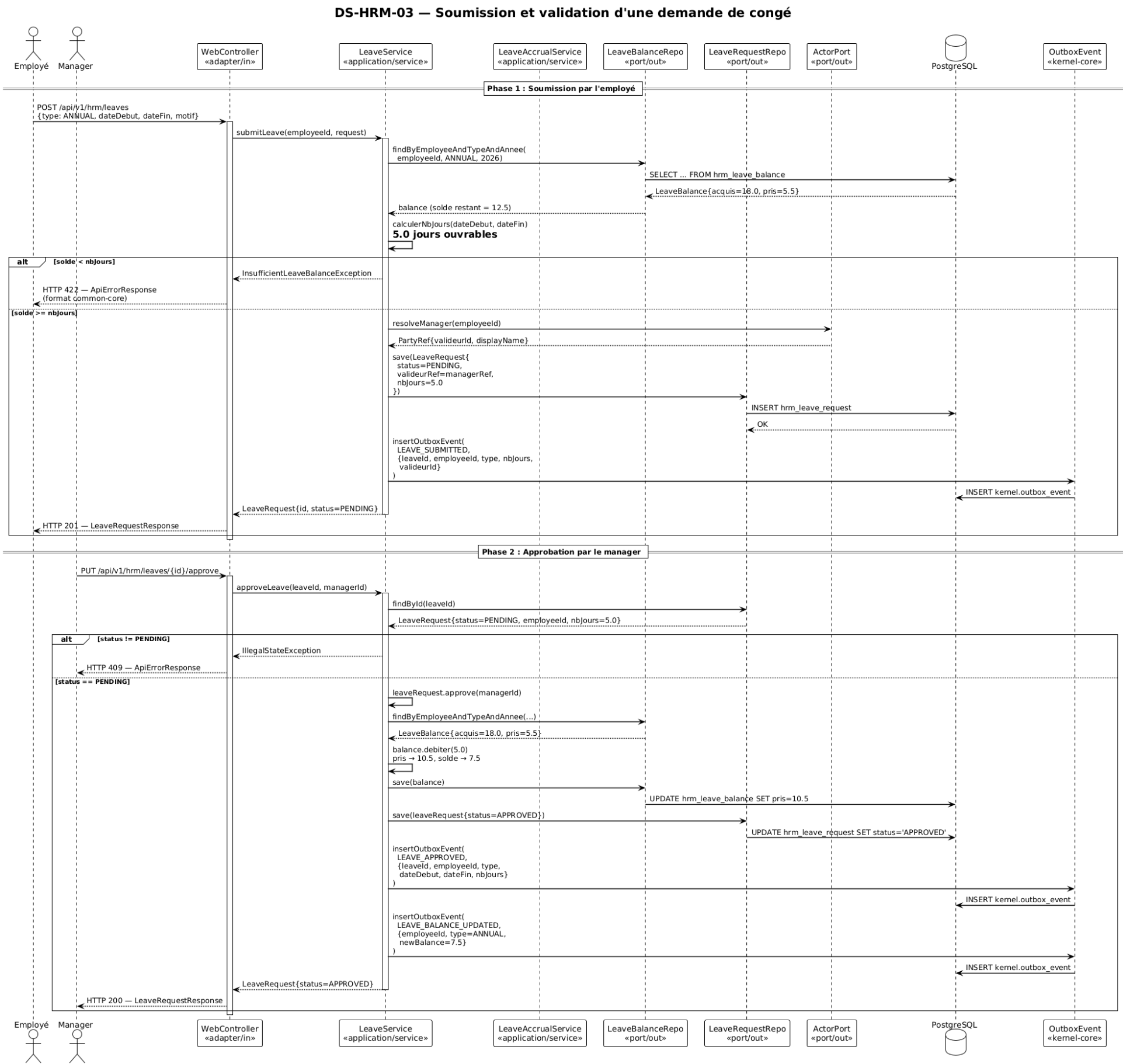


FIGURE 6.3 – DS-HRM-03 Soumission et validation d’une demande de congé

6.3.3 Phase 1 : Soumission par l'employé

1. L'employé envoie `POST /api/v1/hrm/leaves` avec le type (ANNUAL), les dates de début et de fin, et un motif.
2. Le `LeaveService` récupère le solde de congés pour le type et l'année en cours. Exemple : acquis = 18,0 jours, pris = 5,5 jours $\Rightarrow$ solde restant = 12,5 jours.
3. Le nombre de jours ouvrables est calculé entre les dates fournies (ex. 5,0 jours).
4. **Vérification du solde** : si le solde est insuffisant (`solde < nbJours`), une erreur HTTP 422 (`InsufficientLeaveBalanceException`) est retournée au format `ApiErrorResponse` de `common-core`.
5. Le manager de l'employé est résolu via `ActorPort.resolveManager(employeeId)`, qui retourne une `PartyRef` dénormalisée.
6. La `LeaveRequest` est créée au statut `PENDING` avec la référence du valideur.
7. L'événement `LEAVE_SUBMITTED` est publié via l'outbox.
8. La réponse HTTP 201 est retournée.

6.3.4 Phase 2 : Approbation par le manager

1. Le manager envoie `PUT /api/v1/hrm/leaves/{id}/approve`.
2. Le service vérifie que la demande est au statut `PENDING`. Sinon, erreur HTTP 409.
3. La méthode `approve(managerId)` est appelée sur l'agrégat.
4. Le solde de congés est débité : `pris += nbJours` (ex. pris passe de 5,5 à 10,5 ; solde restant = 7,5).
5. Le statut de la demande passe à `APPROVED`.
6. Deux événements sont publiés : `LEAVE_APPROVED` et `LEAVE_BALANCE_UPDATED`.

6.4 DS-HRM-04 : Création d'un employé

6.4.1 Description

Ce scénario détaille la création d'un employé rattaché à un acteur existant dans `actor-core`, incluant la génération du matricule, la création du contrat initial et l'initialisation des soldes de congés.

6.4.2 Diagramme de séquence

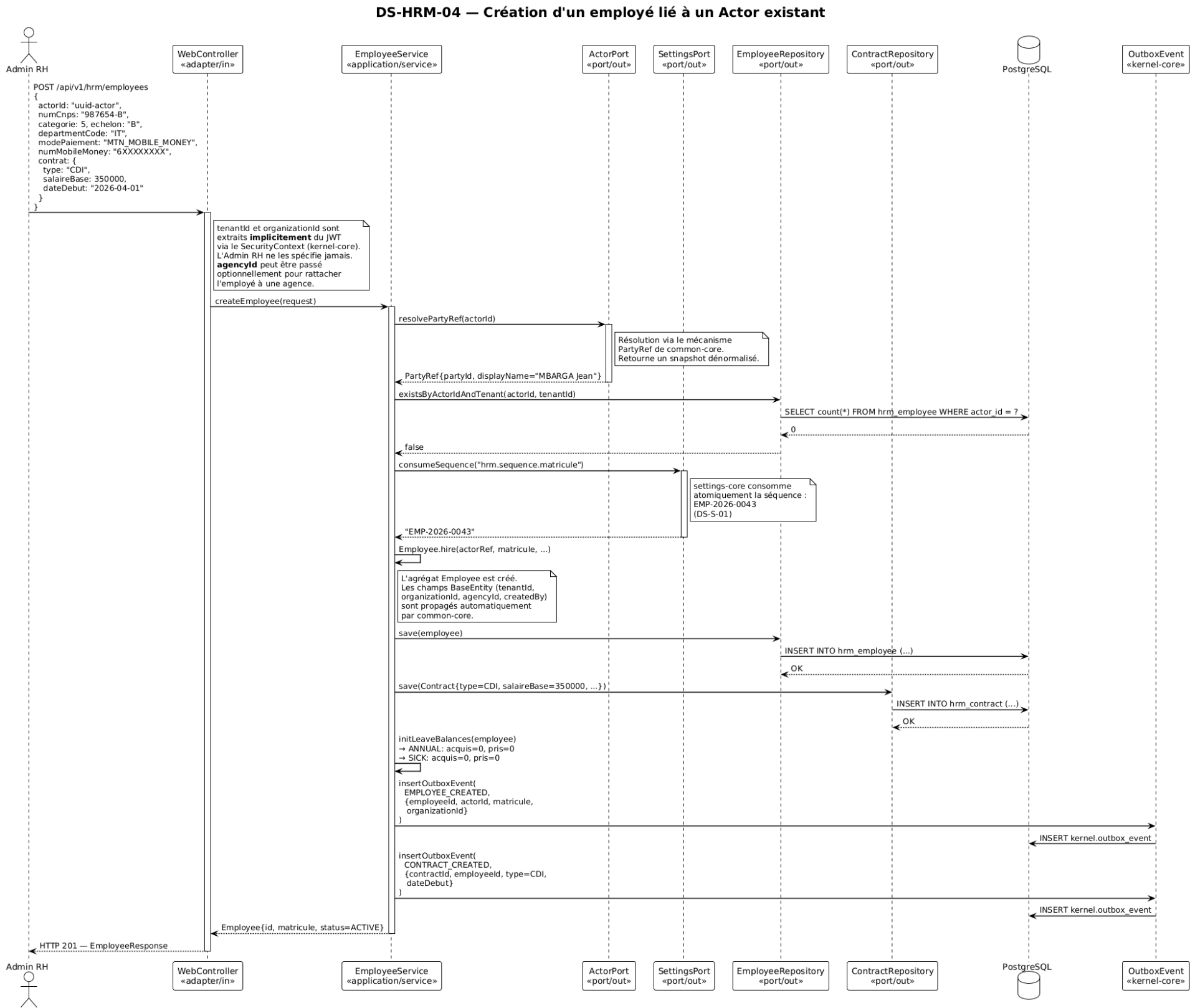


FIGURE 6.4 – DS-HRM-04 Création d'un employé lié à un Actor existant

6.4.3 Description textuelle du flux

1. L'Admin RH envoie `POST /api/v1/hrm/employees` avec l'`actorId`, les données RH (`numCnps`, catégorie, échelon, département, mode de paiement) et le contrat initial (type, salaire de base, date de début).
2. Le `EmployeeService` résout la `PartyRef` via `actor-core` pour vérifier l'existence de l'acteur et récupérer son nom d'affichage.
3. Le service vérifie qu'il n'existe pas déjà un employé pour cet acteur dans ce tenant (`existsByActorIdAndTenant`). En cas de doublon, erreur HTTP 409.
4. Le matricule est généré atomiquement via `settings-core` en consommant la séquence documentaire `hrm.sequence.matricule` (ex. EMP-2026-0043).
5. L'agrégat `Employee` est créé via la méthode fabrique `Employee.hire()`. Les champs `BaseEntity` (`tenantId`, `organizationId`, `agencyId`, `createdBy`) sont propagés automatiquement par `common-core` depuis le `SecurityContext`.
6. Le contrat initial est créé et persisté.
7. Les soldes de congés sont initialisés pour les types ANNUAL et SICK (acquis = 0, pris = 0).
8. Les événements `EMPLOYEE_CREATED` et `CONTRACT_CREATED` sont publiés.
9. La réponse HTTP 201 est retournée.

6.5 DS-HRM-05 : Demande et déduction d'une avance sur salaire

6.5.1 Description

Ce scénario en trois phases couvre le cycle complet d'une avance sur salaire : la demande par l'employé, l'approbation par le comptable, et la déduction automatique lors du calcul de paie.

6.5.2 Diagramme de séquence

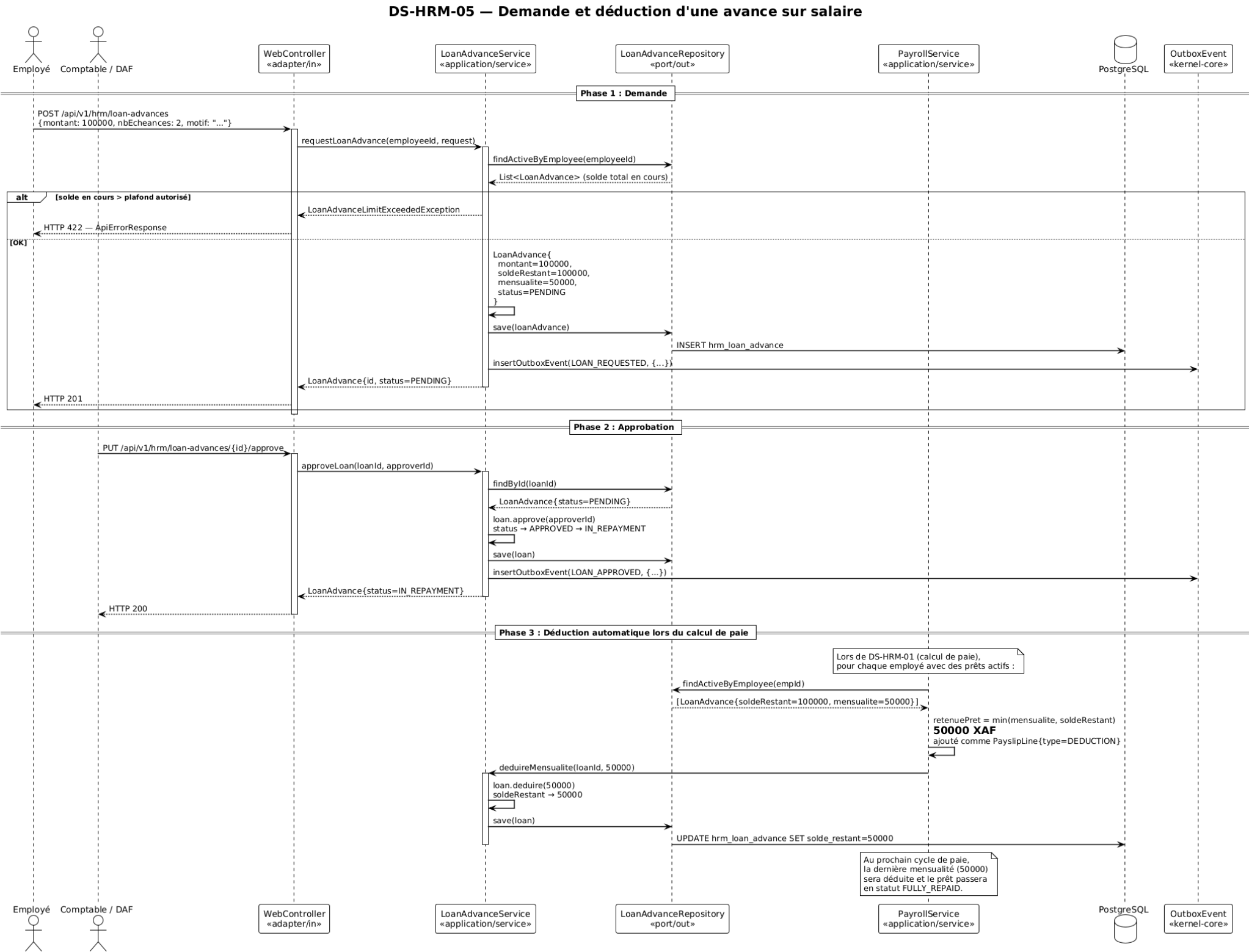


FIGURE 6.5 – DS-HRM-05 Demande et déduction d’une avance sur salaire

6.5.3 Phase 1 : Demande par l'employé

1. L'employé envoie `POST /api/v1/hrm/loan-advances` avec le montant (ex. 100 000 FCFA), le nombre d'échéances (ex. 2) et le motif.
2. Le service vérifie que le cumul des prêts en cours ne dépasse pas le plafond autorisé. En cas de dépassement, erreur HTTP 422.
3. La `LoanAdvance` est créée : `montant = 100 000`, `soldeRestant = 100 000`, `mensualité = 50 000`, `statut = PENDING`.

6.5.4 Phase 2 : Approbation par le comptable

1. Le comptable envoie `PUT /api/v1/hrm/loan-advances/{id}/approve`.
2. Le statut passe de `PENDING` à `IN_REPAYMENT`.
3. L'événement `LOAN_APPROVED` est publié.

6.5.5 Phase 3 : Déduction automatique lors du calcul de paie

1. Lors du calcul de paie (DS-HRM-01), le `PayrollCalculationEngine` récupère les prêts actifs de chaque employé.
2. La retenue est calculée : `min(mensualite, soldeRestant) = 50 000 FCFA`. Elle est ajoutée comme `PayslipLine` de type `DEDUCTION`.
3. Le `LoanAdvanceService` est appelé pour déduire la mensualité : `soldeRestant` passe de 100 000 à 50 000.
4. Au cycle suivant, la dernière mensualité (50 000) est déduite et le prêt passe en statut `FULLY_REPAID`.

6.6 DS-HRM-06 : Acquisition mensuelle automatique des congés

6.6.1 Description

Ce scénario décrit le processus automatisé d'acquisition de congés, exécuté mensuellement par un job planifié (cron le 1^{er} du mois). Il crédite les soldes de congés de chaque employé actif selon les règles légales camerounaises.

6.6.2 Diagramme de séquence

DS-HRM-06 — Acquisition mensuelle automatique des congés

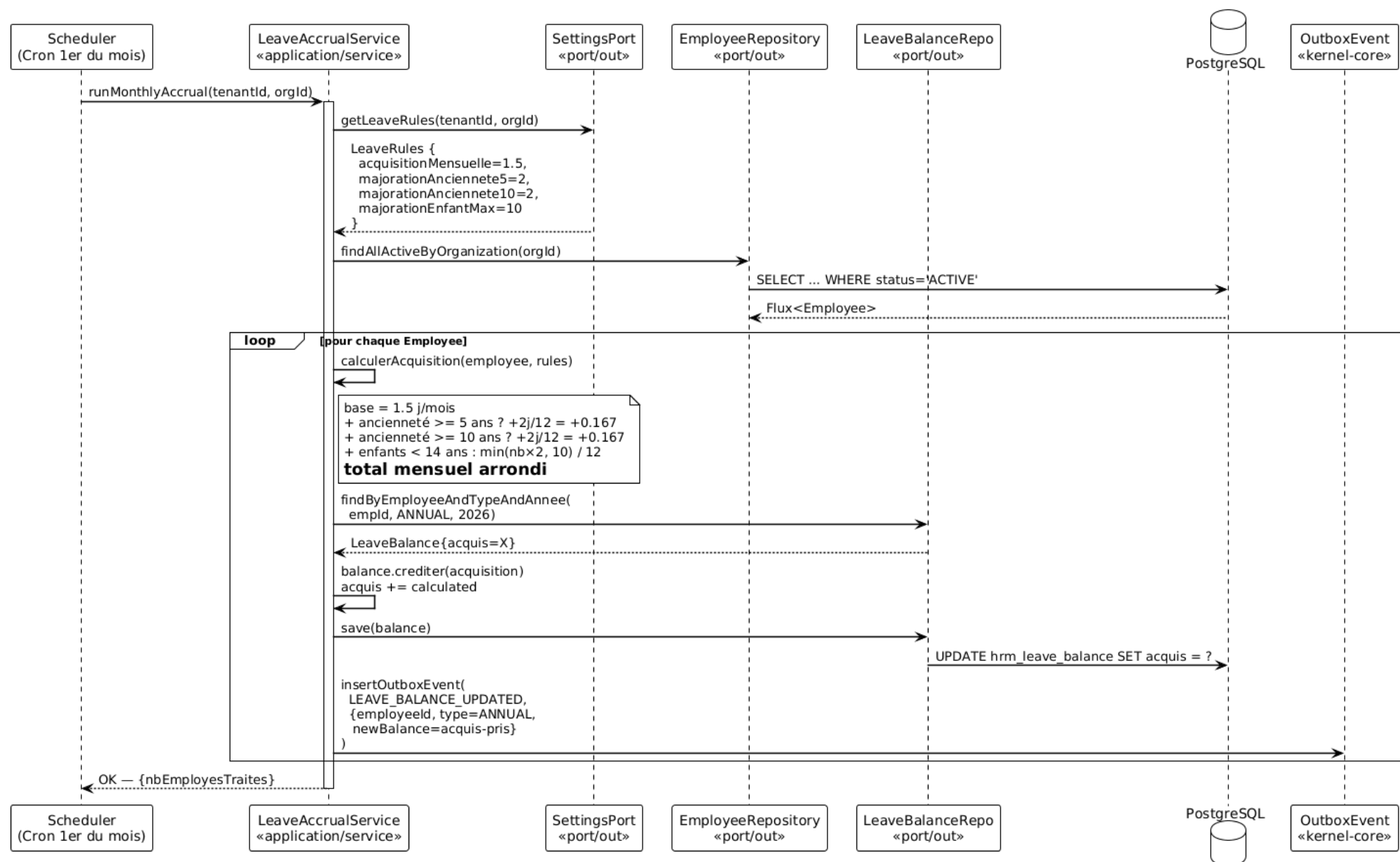


FIGURE 6.6 – DS-HRM-06 Acquisition mensuelle automatique des congés

6.6.3 Description textuelle du flux

1. Le scheduler (cron) déclenche `LeaveAccrualService.runMonthlyAccrual(tenantId, orgId)` le 1^{er} de chaque mois.
2. Les règles d'acquisition sont chargées depuis `settings-core` :
 - Acquisition mensuelle de base : 1,5 jour/mois (soit 18 jours/an).
 - Majoration ancienneté ≥ 5 ans : +2 jours/an (soit +0,167 jour/mois).
 - Majoration ancienneté ≥ 10 ans : +2 jours/an supplémentaires.
 - Majoration pour enfants < 14 ans : $\min(\text{nb_enfants} \times 2, 10) / 12$ jours/-mois.
3. Pour chaque employé actif :
 - a. L'acquisition mensuelle est calculée en fonction de l'ancienneté et des charges de famille.
 - b. Le solde de congés annuels (`LeaveBalance` de type `ANNUAL`) est crédité du montant calculé.
 - c. Un événement `LEAVE_BALANCE_UPDATED` est publié.
4. Le nombre d'employés traités est retourné au scheduler.

7 Diagramme d'activité

7.1 Processus de calcul de paie mensuel

Le diagramme d'activité suivant formalise le processus métier complet de calcul de paie, du point de vue de l'Admin RH et du système `hrm-core`. Il met en évidence les points de décision, les boucles de traitement et les étapes de validation.

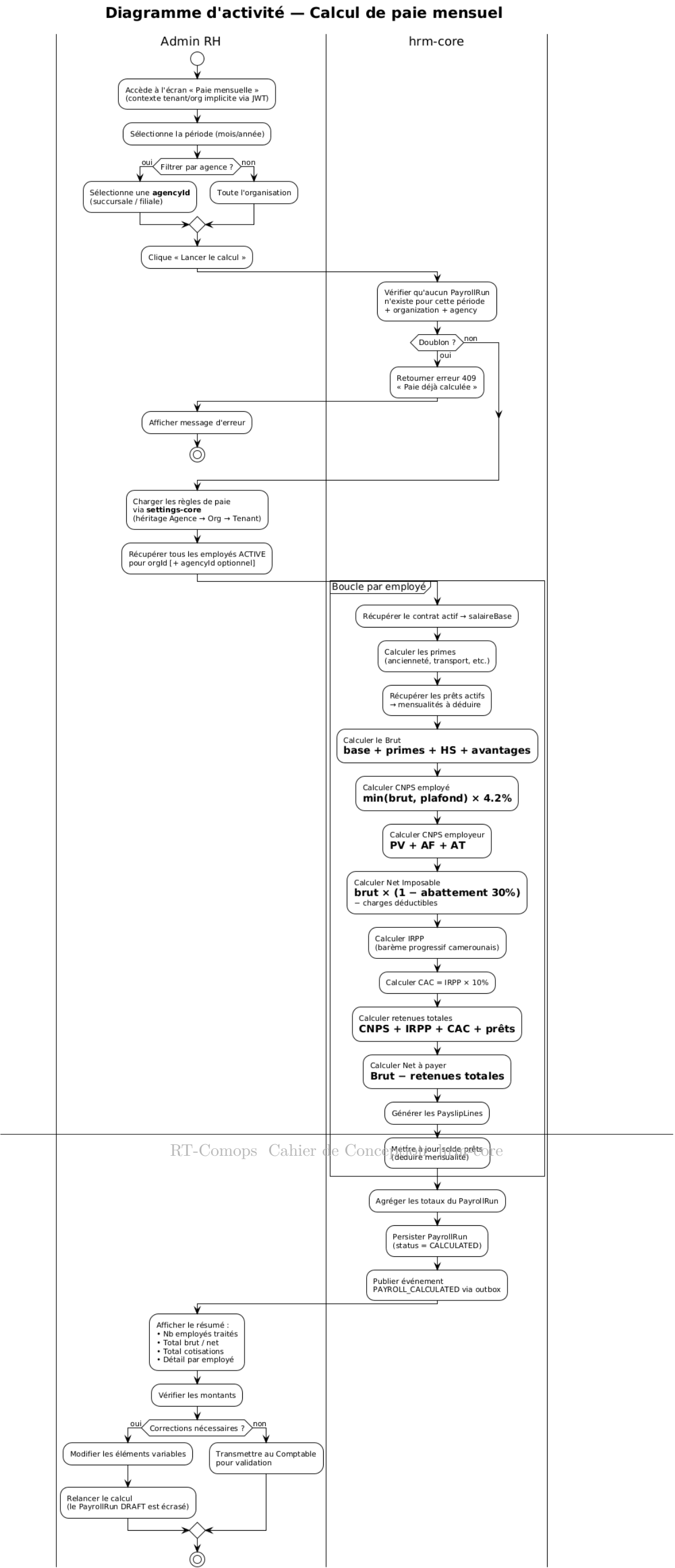


FIGURE 7.1 – Diagramme d'activité Processus de calcul de paie mensuel

7.1.1 Description du processus

1. L'Admin RH accède à l'écran de paie mensuelle. Le contexte tenant/organisation est implicitement déterminé par le JWT.
2. Il sélectionne la période (mois/année) et, optionnellement, une agence pour restreindre le périmètre.
3. Le système vérifie l'absence de doublon pour cette combinaison période/organisation/agence.
4. Les règles de paie sont chargées avec résolution par héritage (Agence → Organisation → Tenant).
5. **Pour chaque employé actif**, le moteur de calcul exécute la séquence de calcul complète (brut, cotisations, IRPP, CAC, prêts, net).
6. Les totaux sont agrégés et le `PayrollRun` est persisté au statut `CALCULATED`.
7. L'Admin RH examine le résumé et décide :
 - Si des corrections sont nécessaires, il modifie les éléments variables et relance le calcul (le `PayrollRun DRAFT` est écrasé).
 - Si les montants sont corrects, il transmet au Comptable pour validation (UC-07).

8 Modèle logique de données

8.1 Vue d'ensemble

Le modèle de données du module `hrm-core` comprend **13 tables**, toutes préfixées par `hrm_` et stockées dans le schéma PostgreSQL propre à chaque tenant. Toutes les entités (sauf `hrm_payslip_line` et `hrm_review_objective` qui sont des objets valeur allégés) portent les champs d'audit hérités de `BaseEntity`.

Modèle Logique de Données — hrm-core (schéma tenant)

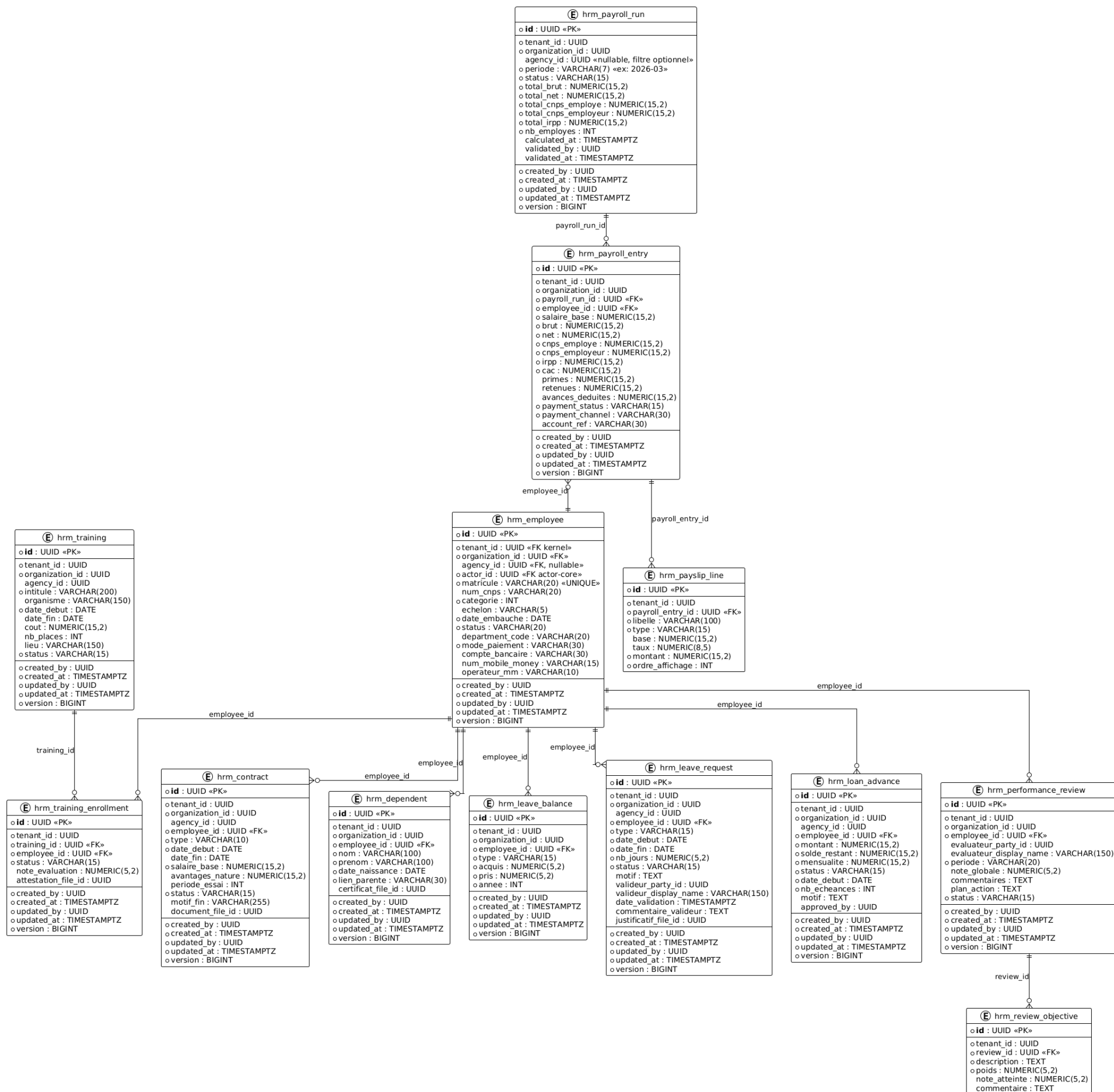


FIGURE 8.1 – Modèle logique de données de hrm-core

8.2 Description des tables

8.2.1 hrm_employee

TABLE 8.1 – Table hrm_employee Données RH de l'employé

Colonne	Type	Description
id	UUID (PK)	Identifiant unique.
tenant_id	UUID (FK)	Référence au tenant (kernel-core).
organization_id	UUID (FK)	Référence à l'organisation.
agency_id	UUID (FK, nullable)	Agence de rattachement (optionnel).
actor_id	UUID (FK)	Référence vers l'acteur dans actor-core.
matricule	VARCHAR(20) UNIQUE	Matricule généré automatiquement (ex. EMP-2026-0043).
num_cnps	VARCHAR(20)	Numéro CNPS de l'employé.
categorie	INT	Catégorie professionnelle.
echelon	VARCHAR(5)	Échelon dans la grille salariale.
date_embauche	DATE	Date d'embauche.
status	VARCHAR(20)	Statut (ACTIVE, ON_LEAVE, SUSPENDED, TERMINATED).
department_code	VARCHAR(20)	Code du département de rattachement.
mode_paiement	VARCHAR(30)	Canal de paiement (BANK_TRANSFER, MTN_MOBILE_MONEY, etc.).
compte_bancaire	VARCHAR(30)	Numéro de compte bancaire.
num_mobile_money	VARCHAR(15)	Numéro de téléphone mobile money.
opérateur_mm	VARCHAR(10)	Opérateur mobile money (MTN, ORANGE).

8.2.2 hrm_contract

TABLE 8.2 – Table hrm_contract Contrats de travail

Colonne	Type	Description
id	UUID (PK)	Identifiant unique.
employee_id	UUID (FK)	Référence vers l'employé.
type	VARCHAR(10)	Type de contrat (CDD, CDI, STAGE, INTERIM).
date_debut	DATE	Date de début du contrat.
date_fin	DATE (nullable)	Date de fin (null pour CDI).
salaire_base	NUMERIC(15,2)	Salaire de base mensuel en FCFA.
avantages_nature	NUMERIC(15,2)	Montant des avantages en nature.
periode_essai	INT	Durée de la période d'essai en mois.
status	VARCHAR(15)	Statut du contrat (ACTIVE, EXPIRED, TERMINATED, RENEWED).
motif_fin	VARCHAR(255)	Motif de fin de contrat.
document_file_id	UUID	Référence vers le fichier du contrat (file-core).

8.2.3 hrm_dependent

TABLE 8.3 – Table hrm_dependent Personnes à charge

Colonne	Type	Description
id	UUID (PK)	Identifiant unique.
employee_id	UUID (FK)	Référence vers l'employé.
nom	VARCHAR(100)	Nom de famille.
prenom	VARCHAR(100)	Prénom.
date_naissance	DATE	Date de naissance.
lien_parente	VARCHAR(30)	Lien de parenté (enfant, conjoint, etc.).
certificat_file_id	UUID	Référence vers le certificat justificatif.

8.2.4 hrm_payroll_run

TABLE 8.4 – Table hrm_payroll_run Exécution de paie

Colonne	Type	Description
id	UUID (PK)	Identifiant unique.
periode	VARCHAR(7)	Période au format YYYY-MM (ex. 2026-03).
status	VARCHAR(15)	Statut (DRAFT, CALCULATED, VALIDATED, PAID).
total_brut	NUMERIC(15,2)	Total brut agrégé.
total_net	NUMERIC(15,2)	Total net agrégé.
total_cnps_employe	NUMERIC(15,2)	Total des cotisations CNPS employé.
total_cnps_employeur	NUMERIC(15,2)	Total des cotisations CNPS employeur.
total_irpp	NUMERIC(15,2)	Total IRPP.
nb_employes	INT	Nombre d'employés traités.
calculated_at	TIMESTAMPTZ	Horodatage du calcul.
validated_by	UUID	Identifiant du valideur.
validated_at	TIMESTAMPTZ	Horodatage de la validation.

8.2.5 hrm_payroll_entry

TABLE 8.5 – Table hrm_payroll_entry Ligne de paie par employé

Colonne	Type	Description
id	UUID (PK)	Identifiant unique.
payroll_run_id	UUID (FK)	Référence vers le PayrollRun.
employee_id	UUID (FK)	Référence vers l'employé.
salaire_base	NUMERIC(15,2)	Salaire de base du contrat.
brut	NUMERIC(15,2)	Salaire brut calculé.
net	NUMERIC(15,2)	Salaire net à payer.
cnps_employe	NUMERIC(15,2)	Cotisation CNPS part salariale.
cnps_employeur	NUMERIC(15,2)	Cotisation CNPS part patronale.
irpp	NUMERIC(15,2)	Impôt sur le revenu des personnes physiques.
cac	NUMERIC(15,2)	Centimes additionnels communaux (10% de l'IRPP).
primes	NUMERIC(15,2)	Total des primes.
retenues	NUMERIC(15,2)	Total des retenues hors prêts.
avances_deduites	NUMERIC(15,2)	Montant des avances/prêts déduits.
payment_status	VARCHAR(15)	Statut de paiement (PENDING, PROCESSING, COMPLETED, FAILED).
payment_channel	VARCHAR(30)	Canal de paiement utilisé.
account_ref	VARCHAR(30)	Référence du compte de paiement.

8.2.6 hrm_payslip_line

TABLE 8.6 – Table hrm_payslip_line Lignes de bulletin de paie

Colonne	Type	Description
id	UUID (PK)	Identifiant unique.
payroll_entry_id	UUID (FK)	Référence vers la PayrollEntry.
libelle	VARCHAR(100)	Libellé de la ligne (ex. 'Salaire de base', 'CNPS').
type	VARCHAR(15)	EARNING (gain) ou DEDUCTION (retenue).
base	NUMERIC(15,2)	Montant de base pour le calcul.
taux	NUMERIC(8,5)	Taux appliqué.
montant	NUMERIC(15,2)	Montant calculé de la ligne.
ordre_affichage	INT	Ordre d'affichage sur le bulletin.

8.2.7 Tables restantes

Les tables `hrm_leave_balance`, `hrm_leave_request`, `hrm_loan_advance`, `hrm_training`, `hrm_training_enrollment`, `hrm_performance_review` et `hrm_review_objective` suivent les mêmes principes de conception. Leurs colonnes correspondent fidèlement aux attributs des classes de domaine décrits au chapitre 4. Les champs d'audit (`created_by`, `created_at`, `updated_by`, `updated_at`, `version`) sont présents sur toutes les entités sauf les objets valeur (`hrm_payslip_line`, `hrm_review_objective`).

8.3 Contraintes d'intégrité

- Toutes les clés primaires sont des UUID générés côté applicatif.
- Le `matricule` de l'employé est unique au niveau du tenant (`UNIQUE`).
- Les clés étrangères (`employee_id`, `payroll_run_id`, etc.) garantissent l'intégrité référentielle.
- Le champ `version` assure le **verrouillage optimiste** : toute modification incrémente la version et échoue si la version en base a changé entre-temps.
- Les montants monétaires utilisent `NUMERIC(15,2)` pour éviter les erreurs d'arrondi des types flottants.
- Les horodatages utilisent `TIMESTAMPZ` pour la gestion correcte des fuseaux horaires.

9 Règles métier et contraintes

9.1 Règles de calcul de paie (législation camerounaise)

Règle métier RM-01 : Calcul de la CNPS (Caisse Nationale de Prévoyance Sociale)

- **Part salariale (Pension Vieillesse)** : $\min(\text{brut}, \text{plafond}) \times 4,2\%$
- **Plafond mensuel (Pension Vieillesse uniquement)** : 750 000 FCFA, paramétrable via `settings-core`.

Règle métier RM-02 : Calcul de l'IRPP (Impôt sur le Revenu des Personnes Physiques)

- *Exonération* : aucune retenue IRPP si `brut_mensuel` < 62 000 FCFA.
- Le net imposable mensuel est calculé ainsi (loi de finances 2024) :
 - Si `brut` ≤ 1 333 333 FCFA : `net_imposable` = `brut` × 0,70 (abattement forfaitaire de 30%)
 - Si `brut` > 1 333 333 FCFA : `net_imposable` = `brut` − 400 000 (abattement plafonné à 400 000 FCFA/mois)
- **Barème progressif mensuel** (paramétrable via `settings-core`) :
 - De 0 à 166 667 FCFA : **10%**
 - De 166 668 à 250 000 FCFA : **15%**
 - De 250 001 à 416 667 FCFA : **25%**
 - Au-delà de 416 667 FCFA : **35%**

Règle métier RM-03 : Centimes Additionnels Communaux (CAC)

Le CAC est calculé comme **10% de l'IRPP** : $cac = irpp \times 0,10$.

Note : le seuil d'exonération de 62 000 FCFA/mois s'applique également au CAC (si IRPP = 0, alors CAC = 0).

Règle métier RM-04 : Déduction automatique des prêts

Lors de chaque calcul de paie, la déduction pour prêt est : $\min(mensualite, soldeRestant)$. Lorsque `soldeRestant` atteint 0, le prêt passe en statut `FULLY_REPAID`.

Règle métier RM-06 : Redevance Audiovisuelle (RAV)

La RAV est due par tous les salariés des secteurs privé, public et parapublic (Ordonnance n° 89/004 du 12 décembre 1989). Elle est retenue à la source par l'employeur et reversée au profit de la CRTV.

- **Base de calcul** : montant brut mensuel des salaires perçus.
- **Barème forfaitaire mensuel** (paramétrable via `settings-core`) :

Salaire brut mensuel (FCFA)	Montant RAV (FCFA)
De 0 à 50 000	0
De 50 001 à 100 000	750
De 100 001 à 200 000	1 950
De 200 001 à 300 000	3 250
De 300 001 à 400 000	4 550
De 400 001 à 500 000	5 850
De 500 001 à 600 000	7 150
De 600 001 à 700 000	8 450
De 700 001 à 800 000	9 750
De 800 001 à 900 000	11 050
De 900 001 à 1 000 000	12 350
Au-delà de 1 000 000	13 000

Règle métier RM-07 : Contribution au Crédit Foncier du Cameroun (CFC)

La CFC est une taxe parafiscale reversée au Crédit Foncier du Cameroun pour financer les projets d'habitat.

- **Part salariale** : $brut \times 1\%$
- **Base de calcul** : montant brut total des rémunérations (y compris avantages en espèces et en nature).

- *Note : le seuil d'exonération de 62 000 FCFA/mois s'applique également à la CFC.*

Règle métier RM-09 : Taxe de Développement Local (TDL)

La TDL est perçue par l'État et reversée aux communes en contrepartie des services de base rendus aux populations (éclairage public, assainissement, enlèvement des ordures, etc.).

- **Base de calcul** : salaire de base mensuel.
- **Barème forfaitaire annuel** (paramétrable via `settings-core`, proratisé mensuellement) :

Salaire de base mensuel (FCFA)	TDL annuelle (FCFA)
De 62 000 à 75 000	3 000
De 75 001 à 100 000	6 000
De 100 001 à 125 000	9 000
De 125 001 à 150 000	12 000
De 150 001 à 200 000	15 000
De 200 001 à 250 000	18 000
De 250 001 à 300 000	24 000
De 300 001 à 500 000	27 000
Supérieure à 500 000	30 000

Note : en pratique, la TDL est souvent prélevée en une seule fois en début d'année ou répartie mensuellement ($tdl_mensuelle = tdl_annuelle / 12$). Le seuil de 62 000 FCFA s'applique également ici.

9.2 Règles d'acquisition des congés

Règle métier RM-05 : Acquisition mensuelle de congés

- **Acquisition de base : 1,5 jour ouvrable par mois** (soit 18 jours ouvrables par an), conformément à l'article 89 du Code du travail (loi n° 92/007 du 14 août 1992).
- **Majoration pour ancienneté** (art. 90 al. 3) : +2 jours ouvrables par tranche entière de 5 ans de service, continue ou non.
 - Ancienneté $\in [5, 10[$ ans : **+2 jours/an** (soit +0,167 jour/mois)
 - Ancienneté $\in [10, 15[$ ans : **+4 jours/an** (soit +0,333 jour/mois)
 - Ancienneté $\in [15, 20[$ ans : **+6 jours/an** (soit +0,500 jour/mois)

- Et ainsi de suite par palier de 5 ans.
- Formule générale : $\text{maj_anciennete} = \text{floor}(\text{anciennete} / 5) \times 2 \text{ jours/an.}$
- **Majoration pour enfants à charge** (art. 90 al. 2 *mères salariées uniquement*) :
 - Condition : enfant âgé de **moins de 6 ans** à la date de départ en congé, inscrit à l'état civil et vivant au foyer.
 - Taux : +2 jours ouvrables par enfant éligible.
 - Exception : si le congé principal n'excède pas 6 jours, la majoration est réduite à +1 jour par enfant.
 - Formule mensuelle : $\text{maj_enfants} = \text{nb_enfants_moins_6ans} \times 2 / 12 \text{ jours/mois}$ (plafond selon convention collective applicable).
- **Cas particulier jeunes travailleurs** (art. 90 al. 1) : pour les salariées de **moins de 18 ans**, l'acquisition est portée à **2,5 jours ouvrables/mois** (30 jours/an).
- L'acquisition est exécutée automatiquement le 1^{er} de chaque mois par un job planifié. Les majorations sont recalculées à chaque exécution sur la base de l'ancienneté et de la situation familiale à jour.

9.3 Paramètres de configuration (settings-core)

9.3.1 Paramètres CNPS

Paramètre	Type	Valeur par défaut	Description
cnps_plafond_mensuel	Nombre	750 000 FCFA	Plafond mensuel base cotisable (P
cnps_taux_pv_salarial	Pourcentage	4,2%	Taux Pension Vieillesse part salar

9.3.2 Paramètres IRPP

Paramètre	Type	Valeur par défaut	Description
irpp_seuil_exoneration	Nombre	62 000 FCFA	Brut mensuel en dess
irpp_seuil_plafond_abattement	Nombre	1 333 333 FCFA	Seuil de plafonnement
irpp_abattement_taux	Pourcentage	30%	Taux d'abattement fo
irpp_abattement_plafond_mensuel	Nombre	400 000 FCFA	Plafond mensuel de l'
irpp_tranches	Tableau	voir ci-dessous	Barème progressif me

Règle métier **Détail : irpp_tranches** **Barème progressif mensuel**

Tranche	De (FCFA)	À (FCFA)	Taux
T1	0	166 667	10%
T2	166 668	250 000	15%
T3	250 001	416 667	25%
T4	416 668	$+\infty$	35%

9.3.3 Paramètres CAC

Paramètre	Type	Valeur par défaut	Description
cac_taux	Pourcentage	10%	Taux CAC appliqué sur l'IRPP

9.3.4 Paramètres RAV

Paramètre	Type	Valeur par défaut	Description
rav_tranches	Tableau	voir ci-dessous	Barème forfaitaire mensuel selon le brut

Règle métier Détail : rav_tranches Barème forfaitaire mensuel

De (FCFA)	À (FCFA)	Montant RAV (FCFA)
0	50 000	0
50 001	100 000	750
100 001	200 000	1 950
200 001	300 000	3 250
300 001	400 000	4 550
400 001	500 000	5 850
500 001	600 000	7 150
600 001	700 000	8 450
700 001	800 000	9 750
800 001	900 000	11 050
900 001	1 000 000	12 350
1 000 001	$+\infty$	13 000

9.3.5 Paramètres CFC

Paramètre	Type	Valeur par défaut	Description
cfc_taux_salarial	Pourcentage	1%	Taux CFC part salariale appliqué sur le b

9.3.6 Paramètres TDL

Paramètre	Type	Valeur par défaut	Description
tdl_tranches	Tableau	voir ci-dessous	Barème forfaitaire annuel selon le salaire
tdl_mode_prelevement	Enum	MENSUEL	MENSUEL (œ12/mois) ou ANNUEL (en jan)

Règle métier Détail : tdl_tranches Barème forfaitaire

De (FCFA)	À (FCFA)	TDL annuelle (FCFA)	TDL mensuelle (FCFA)
0	61 999	0	0
62 000	75 000	3 000	250
75 001	100 000	6 000	500
100 001	125 000	9 000	750
125 001	150 000	12 000	1 000
150 001	200 000	15 000	1 250
200 001	250 000	18 000	1 500
250 001	300 000	24 000	2 000
300 001	500 000	27 000	2 250
500 001	+∞	30 000	2 500

9.3.7 Paramètres Congés

Paramètre	Type	Valeur par défaut	Description
conge_base_jours_mois	Nombre	1,5	Acquisition de base (j)
conge_jeune_travailleur_jours_mois	Nombre	2,5	Acquisition pour les m
conge_age_limite_jeune	Nombre	18	Âge limite régime jeun
conge_anciennete_palier_ans	Nombre	5	Palier d'ancienneté en
conge_anciennete_jours_par_palier	Nombre	2	Jours supplémentaires
conge_enfant_age_limite	Nombre	6	Âge limite enfants (mè
conge_enfant_jours_par_enfant	Nombre	2	Jours supplémentaires

9.4 Contraintes métier transverses

Règle métier RM-06 : Unicité de la paie

Un seul PayrollRun peut exister pour une combinaison donnée de (période, organizationId, agencyId). Toute tentative de doublon est rejetée avec une erreur HTTP 409.

Règle métier RM-07 : Unicité de l'employé par acteur

Un seul `Employee` peut exister pour un `actorId` donné au sein d'un tenant. Cela évite les doublons lorsqu'un acteur est déjà enregistré.

Règle métier RM-08 : Contrat actif unique

Un employé ne peut avoir qu'un seul contrat au statut `ACTIVE` à un instant donné.

Règle métier RM-09 : Vérification du solde de congés

Une demande de congé ne peut être soumise que si le solde restant (acquis – pris) est supérieur ou égal au nombre de jours demandés. En cas de solde insuffisant, l'erreur `InsufficientLeaveBalanceException` (HTTP 422) est retournée.

Règle métier RM-10 : Plafond de prêts

Le cumul des soldes restants des prêts actifs d'un employé ne doit pas dépasser un plafond paramétrable. Ce plafond est vérifié lors de chaque nouvelle demande d'avance.

10 Glossaire

Actor	Personne physique ou morale enregistrée dans <code>actor-core</code> . Un employé est lié à un acteur.
Agence	Niveau le plus fin de la hiérarchie organisationnelle (succursale, filiale). Filtre optionnel.
BaseEntity	Classe abstraite de <code>common-core</code> fournissant les champs <code>id</code> , <code>tenantId</code> , <code>organizationId</code> , <code>audit</code> et <code>version</code> .
CAC	Centimes Additionnels Communaux surtaxe communale égale à 10% de l'IRPP.
CDD	Contrat à Durée Déterminée.
CDI	Contrat à Durée Indéterminée.
CNPS	Caisse Nationale de Prévoyance Sociale (organisme camerounais de sécurité sociale).
Core	Module fonctionnel autonome au sein de RT-Comops (ex. <code>hrm-core</code> , <code>actor-core</code>).
FCFA	Franc CFA monnaie utilisée dans les calculs de paie.
IRPP	Impôt sur le Revenu des Personnes Physiques.
Outbox	Pattern de publication fiable d'événements : l'événement est inséré dans une table dédiée au sein de la même transaction que la modification métier, puis relayé vers Kafka par un scheduler.
PartyRef	Référence dénormalisée vers un acteur (<code>partyId</code> + <code>displayName</code>), définie dans <code>common-core</code> .
PayrollRun	Exécution d'un cycle de paie pour une période, une organisation et optionnellement une agence.
PayslipLine	Ligne de bulletin de paie (gain ou retenue) objet valeur.
R2DBC	Reactive Relational Database Connectivity pilote réactif pour l'accès à PostgreSQL.
Tenant	Locataire du système. Chaque tenant possède un schéma PostgreSQL dédié.

WebFlux Module réactif de Spring Framework pour le traitement non-bloquant des requêtes HTTP.